



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE CIÊNCIAS EXATAS E DA TERRA
DEPARTAMENTO DE INFORMÁTICA E MATEMÁTICA APLICADA
PROGRAMA DE PÓS-GRADUAÇÃO EM SISTEMAS E COMPUTAÇÃO
MESTRADO ACADÊMICO EM SISTEMAS E COMPUTAÇÃO



Uma abordagem para análise de cobertura de código em cenários de evolução

Fladson Thiago Oliveira Gomes

Natal-RN
Fevereiro/2016

Fladson Thiago Oliveira Gomes

Uma abordagem para análise de cobertura de código em cenários de evolução

Dissertação de Mestrado apresentada ao Programa de Pós-Graduação em Sistemas e Computação do Departamento de Informática e Matemática Aplicada da Universidade Federal do Rio Grande do Norte como requisito parcial para a obtenção do grau de Mestre em Sistemas e Computação.

Linha de pesquisa:
Engenharia de Software

Orientador

Prof. Dr. Uirá Kulesza

PPGSC – PROGRAMA DE PÓS-GRADUAÇÃO EM SISTEMAS E COMPUTAÇÃO
DIMAP – DEPARTAMENTO DE INFORMÁTICA E MATEMÁTICA APLICADA
CCET – CENTRO DE CIÊNCIAS EXATAS E DA TERRA
UFRN – UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Natal-RN

Julho/2015

Catálogo da Publicação na Fonte. UFRN / SISBI / Biblioteca Setorial
Especializada do Centro de Ciências Exatas e da Terra – CCET.

Gomes, Fladson Thiago Oliveira.

Uma abordagem para análise de cobertura de código em cenários de evolução /
Fladson Thiago Oliveira Gomes. – Natal, RN, 2016.

66 f. : il.

Orientador: Prof. Dr. Uirá Kulesza.

Dissertação (Mestrado) – Universidade Federal do Rio Grande do Norte. Centro
de Ciências Exatas e da Terra. Departamento de Informática e Matemática Aplicada.
Programa de Pós-Graduação em Sistemas e Computação.

1. Engenharia de software – Dissertação. 2. Evolução de software – Dissertação.
3. Análise de impacto de mudanças – Dissertação. 4. Análise de código –
Dissertação. 5. Cobertura de código – Dissertação. 6. Fluxos de execução –
Dissertação. I. Kulesza, Uirá. II. Título.

RN/UF/BSE-CCET

CDU 004.41

Isto é a vida; Não há planger, nem imprecar, mas aceitar as coisas integralmente, com seus ônus e percalços, glórias e desdouros, e ir por diante.

Machado de Assis

Uma abordagem para análise de cobertura de código em cenários de evolução

Autor: Fladson Thiago Oliveira Gomes

Orientador(a): Prof. Dr. Uirá Kulesza

RESUMO

Atualmente, a etapa de testes no processo de desenvolvimento de software tornou-se imprescindível para garantir a confiabilidade e qualidade do código em produção. As constantes evoluções na arquitetura e código de um sistema, criam sérios desafios para os desenvolvedores e testadores, uma vez que modificações podem não se comportar como o esperado. Neste contexto surge a necessidade de ferramentas e mecanismos que diminuam o impacto negativo gerado pelas constantes evoluções do sistema. Dentre as ferramentas que analisam esse impacto, poucas apresentam os fluxos de execução entre métodos que foram afetados e nenhuma apresenta como resultado se esses fluxos afetados pela evolução estão ou não cobertos pelos testes. Assim, este trabalho apresenta uma abordagem que tem como objetivo principal: (i) analisar a cobertura de código levando em consideração os fluxos de chamadas existentes no sistema que foram afetados por evoluções de código, assim como os fluxos de execução oriundos da execução dos testes; (ii) indicar quais fluxos de chamadas do sistema que possuem métodos modificados e não estão sendo cobertos pelos testes atualmente e que, portanto, poderiam ser considerados para melhorar a qualidade dos testes; e (iii) indicar se houve degradação na qualidade da suíte de testes. Um estudo empírico foi realizado em 6 sistemas e os resultados mostram que a abordagem conseguiu identificar entre 19% e 92% de fluxos de execução afetados por mudanças que não estão cobertos e ainda que 3 dos 6 sistemas tiveram uma degradação na qualidade dos testes.

Palavras-chave: Evolução de Software; Análise de Impacto de Mudanças; Análise de Código; Cobertura de Código; Fluxos de Execução.

An approach to analyse code coverage in evolving scenarios

Author: Fladson Thiago Oliveira Gomes

Supervisor: Prof. Dr. Uirá Kulesza

ABSTRACT

Nowadays, the test activities in the software development process became fundamental to ensure the reliability and quality of software systems code. The frequent evolution in the system's architecture and code creates strong challenges to the developers and testers since the modifications cannot behave as expected. In this context, there is a need of tools and mechanisms that decrease the negative impact generated by these frequent system evolution. There are a few existing tools that shows the execution flows of methods that are affected by the evolutions, however most of them do not show if those affected flows are or not covered by tests. This work presents an approach that has as the main goals: (i) to analyze the code coverage taking into consideration the existing system execution flows that were affected by system evolution; (ii) to indicate which system execution flows that have changed methods and are not covered by the existing automated tests and, therefore, could be considered to improve the system quality through the test regression; and (iii) to indicate if there is a test quality degradation in terms of non-covered execution flows. Our work has also performed an empirical study that analyzes 6 open-source systems using the proposed approach. The study results identified that between 19% and 92% execution flows of those open-source systems are affected by code evolution changes and are not covered by automated tests. In addition, it was detected that 3 of the 6 systems had a test quality degradation in terms of non-covered execution flows.

Keywords: Software Evolution; Change Impact Analysis; Code Analysis; Code Coverage; Execution paths/flows.

Lista de figuras

1	Modelo V em testes de software	p. 22
2	Exemplo de Ramificação	p. 24
3	Fluxos de execução	p. 25
4	Etapa de construção do grafo de chamadas da aplicação	p. 27
5	Etapa de construção do fluxo de chamadas dos testes	p. 28
6	Etapa de análise e extração das evoluções	p. 29
7	Etapa de análise dos fluxos e obtenção dos resultados	p. 30
9	Fluxo de Chamadas do Sistema Simulado	p. 32
10	Fluxos de Execução Afetados pela Mudança no Cenário 1	p. 33
11	Fluxos de Execução Afetados pela Mudança no Cenário 2	p. 34
8	Arquitetura da Abordagem	p. 35
12	Evolução de Cobertura do Sistema Bukkit	p. 43
13	Diagrama de Impacto do Sistema Bukkit	p. 44
14	Evolução de Cobertura do Sistema Cool Reader	p. 45
15	Diagrama de Impacto do Sistema Cool Reader	p. 46
16	Evolução de Cobertura do Sistema Google Auth Library	p. 47
17	Diagrama de Impacto do Sistema Google Auth Library	p. 47
18	Evolução de Cobertura do Sistema GUY	p. 48
19	Diagrama de Impacto do Sistema GUY	p. 49
20	Evolução de Cobertura do Sistema Mamute	p. 50
21	Diagrama de Impacto do Sistema Mamute	p. 51
22	Evolução de Cobertura do Sistema VRaptor	p. 52

23	Diagrama de Impacto do Sistema VRaptor	p. 53
----	--	-------

Lista de tabelas

1	Resultados da análise	p. 41
2	Resultados da análise pra cada Cenário de Evolução do Sistema Bukkit	p. 43
3	Resultados da análise pra cada Cenário de Evolução do Sistema Cool Reader	p. 44
4	Resultados da análise pra cada Cenário de Evolução do Sistema Google Auth Library	p. 46
5	Resultados da análise pra cada Cenário de Evolução do Sistema GUJ .	p. 48
6	Resultados da análise pra cada Cenário de Evolução do Sistema Mamute	p. 49
7	Resultados da análise pra cada Cenário de Evolução do Sistema VRaptor	p. 52

Sumário

1	Introdução	p. 10
1.1	Problema abordado	p. 11
1.2	Limitações de Trabalhos Existentes	p. 12
1.3	Objetivos	p. 15
1.4	Organização do trabalho	p. 16
2	Fundamentação teórica	p. 17
2.1	Análise do Impacto de Mudanças	p. 17
2.1.1	Estratégias para Análise de Impacto	p. 18
2.1.2	Análise de Impacto Estática	p. 20
2.1.3	Análise de Impacto Dinâmica	p. 20
2.2	Cobertura de código	p. 21
2.3	Critérios de cobertura	p. 22
2.3.0.1	Cobertura de Segmento	p. 23
2.3.0.2	Cobertura de Decisão	p. 23
2.3.0.3	Cobertura de Fluxo	p. 24
2.3.0.4	Cobertura de Fluxo de Chamadas Entre Métodos . . .	p. 25
3	Uma abordagem para análise de cobertura de código em cenários de evolução	p. 26
3.1	Visão Geral da Abordagem	p. 26
3.2	Arquitetura da Ferramenta	p. 30
3.3	Demonstração da Abordagem em um Ambiente Controlado	p. 31

4	Avaliação da Abordagem Proposta	p. 36
4.1	Configuração do Estudo	p. 36
4.1.1	Objetivos e Questões de Pesquisa	p. 36
4.1.2	Fases do estudo	p. 37
4.1.2.1	Fase 1 - Seleção dos Sistemas e Evoluções	p. 37
4.1.2.2	Fase 2 - Execução da Mineração de Mudanças e das Análises Estática e Dinâmica	p. 38
4.1.2.3	Fase 3 - Análise dos Resultados	p. 39
4.1.3	Sistemas Analisados	p. 39
4.1.4	Métricas para Avaliação	p. 40
4.2	Resultados Obtidos	p. 41
4.2.1	Visão Geral dos Resultados	p. 41
4.2.2	Resultados e Análise Detalhada de cada Sistema	p. 42
5	Trabalhos Relacionados	p. 54
5.1	Abordagens de Cobertura e Análise de Código	p. 54
5.2	Abordagens de Análise do Impacto de Mudanças	p. 55
5.3	Abordagens de Testes de Regressão	p. 57
6	Conclusão	p. 59
6.1	Principais Contribuições	p. 59
6.2	Limitações da Abordagem	p. 60
6.3	Trabalhos Futuros	p. 61
	Referências	p. 62
	Apêndice A – Formato dos Fluxos de Execução	p. 66

1 Introdução

As constantes evoluções na arquitetura e código de um sistema de software durante seu ciclo de vida, criam sérios desafios para os desenvolvedores e testadores, uma vez que modificações podem não se comportar como o esperado, podendo introduzir diversos efeitos colaterais não desejados. Quando algo é modificado, os desenvolvedores devem checar a corretude dessas mudanças, identificando e analisando os efeitos provocados no comportamento do software. Muitas vezes, uma análise voltada exclusivamente aos componentes modificados é insuficiente e imprecisa para determinar a corretude, uma vez que os efeitos da mudança podem afetar outros componentes do software que dependem daquele que foi modificado.

Para garantir a qualidade do código em constante evolução e poder fazer essa análise de corretude de forma automatizada, as equipes de desenvolvimento têm buscado adotar métodos e técnicas de testes no decorrer do desenvolvimento do software. No entanto, é impraticável que todas as funcionalidades do sistema sejam testadas, considerando os diferentes cenários e possibilidades envolvidas. Dessa forma, é necessário um *trade-off* entre o custo da implementação dos testes e a quantidade de falhas que os testes podem encontrar. A efetividade do teste geralmente é mensurada pela qualidade da suíte de testes, ou seja, a capacidade da suíte revelar falhas no sistema. Uma suíte que revele mais falhas é tida como de maior qualidade se comparada com uma suíte que revele menos falhas (ALEMERIEN; MAGEL, 2014; ALEMERIAN, 2013; KOCHHAR; THUNG; LO, 2015; SHAHID; IBRAHIM, 2011a).

As suítes de teste devem acompanhar a evolução do software, caso contrário mudanças de comportamento não intencionais decorrentes das modificações podem não ser observadas. A suíte de teste pode ser executada com sucesso após mudanças no código e ainda assim é possível que a suíte não tenha sido capaz de identificar efeitos colaterais importantes, como mudanças no fluxo de controle e/ou execução.

Neste contexto surge a necessidade de ferramentas e mecanismos que diminuam o im-

pacto negativo gerado pelas constantes evoluções de software. Um dos mecanismos mais apropriados e que é amplamente adotado em cenários de evolução são os testes de regressão, que consiste em um re-teste seletivo do sistema ou de componentes para verificar que modificações não causaram efeitos indesejados, e que o sistema ou componente continua cumprindo com seus requisitos específicos (IEEE, 1990).

Grande parte dos trabalhos na área de testes de regressão tem focado na eficiência desta atividade no que diz respeito à (i) seleção de um sub conjunto de testes que precisam ser re-executados, (ii) priorização de casos de teste para encontrar erros mais cedo, e (iii) redução do tamanho da suíte de teste de acordo com algum critério, ou seja, a remoção de casos de teste desnecessários (ENGSTRÖM; RUNESON; SKOGLUND, 2010). No entanto, pouca atenção vem sendo dada à eficácia dos testes de regressão, ou seja, além da identificação de quais casos de teste foram afetados pelas mudanças, é necessário um entendimento de como as mudanças afetam o comportamento do software. Encontrar o máximo de diferenças possíveis no comportamento que foram causadas pelas evoluções é uma necessidade para determinar se essas evoluções estão corretas ou não.

Outra área de estudo que investiga o efeito das evoluções é a análise do impacto de mudanças, que identifica partes do software que foram potencialmente afetadas por uma mudança, seja na forma de linhas de código, métodos ou classes (LI et al., 2013). Por fim, existe também técnicas baseadas na cobertura do código, que indicam a quantidade de linhas, decisões, condições, métodos e fluxos (de controle ou de dados) que estão sendo executados pela suíte de testes (NTAFOS, 1988; CORNETT, 2002). A cobertura de código permite que a equipe de desenvolvimento tenha uma ideia da amplitude do código que está sendo executado pela suíte de testes, revelando quais partes do código não estão cobertas e conseqüentemente estariam mais sujeitas à falhas, uma vez que não há testes para verificar tais partes.

1.1 Problema abordado

Tendo em vista a importância de analisar o efeito das mudanças durante a evolução de software, várias pesquisas sobre o assunto vêm sendo realizadas. No entanto, segundo trabalhos de sumarização das áreas (LI et al., 2013; LEHNERT, 2011; SHAHID; IBRAHIM, 2011a; ALEMERIAN, 2013; YANG; LI; WEISS, 2009; ALEMERIEN; MAGEL, 2014; ENGSTRÖM; RUNESON; SKOGLUND, 2010), uma observação pode ser feita no que diz respeito aos resultados apresentados pelas abordagens existentes. Além da identificação das partes

afetadas pelas mudanças (código ou testes), a propagação do impacto (i.e., como mudanças afetam o fluxo de execução do software) também deve ser analisada.

Segundo (LI et al., 2013) e (LEHNERT, 2011), dentre as diversas técnicas de análise de impacto, há uma escassez de técnicas e ferramentas que apresentem como resultado quais os fluxos de execução (*execution traces*) que foram afetados pelas mudanças. Grande parte das técnicas e ferramentas de análise de impacto de mudanças provêm como resultado um coeficiente de impacto ou apenas quais partes isoladas do sistema que foram afetadas. Na área de testes de regressão (ENGSTRÖM; RUNESON; SKOGLUND, 2010) os estudos que analisam o impacto de mudanças têm como objetivo principal selecionar ou prover novos casos de teste que cubram as partes do sistema afetadas pelas mudanças.

Dentre as diversas abordagens existentes na literatura que analisam o impacto de mudanças em cenários de evolução, poucas apresentam explicitamente quais fluxos de execução foram afetados por mudanças, a grande maioria tem como objetivo apresentar quais testes foram afetados (que precisariam ser re-testados ou atualizados, se necessário) ou quais outros módulos do código foram impactados.

1.2 Limitações de Trabalhos Existentes

O trabalho de Law e Rothermel (LAW; ROTHERMEL, 2003) é um dos trabalhos que introduziram a análise de fluxos de execução na área de análise de impacto. A abordagem, intitulada PATHIMPACT, captura os fluxos de execução a partir de código instrumentado (análise dinâmica) utilizando a técnica de *forward slicing*.

A técnica de *forward slicing* é definida, por (LAW; ROTHERMEL, 2003), da seguinte forma: se um segmento s^1 é modificado, qualquer segmento que é executado após s e esteja na pilha de execução após o retorno do método de s , é incluído no conjunto de segmentos potencialmente impactados pelas mudanças. Diferente da técnica de *backward slicing*, que é definida da seguinte forma: se um segmento s é modificado, todos os fluxos de execução que passam por s são incluídos no conjunto de fluxos potencialmente impactados pelas mudanças (TIP, 1995).

A abordagem de (LAW; ROTHERMEL, 2003) utiliza a técnica de *forward slicing*, diferente da abordagem aqui proposta, que utiliza a técnica de *backward slicing*, e como limitação, podemos citar que o trabalho de (LAW; ROTHERMEL, 2003) não utiliza a análise estática para determinar o conjunto de fluxos do sistema, ou seja, o conjunto de fluxos

¹Linha de código que executa ou modifica o estado do programa

extraídos pela análise é limitado ao código que foi instrumentado. Além disso a abordagem de (LAW; ROTHERMEL, 2003) tem como principal objetivo prever quais fluxos de execução serão afetados por modificações no código, ou seja, a abordagem não busca determinar se esse conjunto está sendo executado pelos testes.

A abordagem apresentada em (ORSO; APIWATTANAPONG; HARROLD, 2003), realiza a análise de mudanças no nível de método, e determina, a partir de uma combinação das análises de código estática e dinâmica, quais fluxos de execução foram impactados pelas mudanças e ainda conseguem identificar se os mesmos estão ou não sendo testados. No entanto, o estudo difere deste em alguns pontos importantes: (i) a identificação do conjunto de fluxos de execução afetados pelas mudanças é realizada a partir de uma análise dinâmica, ou seja, é necessário que haja uma instrumentação específica no código e que o sistema seja executado por algum usuário, o que restringe a análise à apenas o código que foi executado; (ii) o conjunto de fluxos de execução afetados que são identificados pela abordagem de (ORSO; APIWATTANAPONG; HARROLD, 2003) é obtido a partir da técnica de *forward slicing*; (iii) por último, o trabalho é voltado para a etapa de produção (*in field*), ou seja, tem como objetivo analisar apenas o que é executado pelo usuário.

Outros trabalhos seguiram a linha de estudo introduzida por Law e Rothermel (LAW; ROTHERMEL, 2003) e desenvolveram abordagens que também apresentam quais fluxos de execução foram impactados por mudanças, apresentando uma maior precisão e/ou um menor custo ou utilizando outros tipos de análise (BREECH; TEGTMEYER; POLLOCK, 2006; CAI; SANTELICES, 2014; SUN et al., 2010; APIWATTANAPONG; ORSO; HARROLD, 2005; SUN et al., 2011). Todavia, nenhuma dessas abordagens busca determinar se os fluxos de execução classificados como impactados estão sendo acessados ou não pela suíte de testes.

Na área de testes de regressão a literatura é extensa, mas a maioria dos trabalhos tem como foco principal a eficiência na re-execução da suíte de testes. Chianti (REN et al., 2004) é uma ferramenta que analisa qual o impacto que mudanças, realizadas entre duas versões de código, possuem sobre os testes, ou seja, como resultado, a ferramenta apresenta quais foram os testes que tiveram seu comportamento afetado tendo em vista as diferenças no código. A abordagem adotada na ferramenta utiliza a construção de grafos de chamadas e análise atômica de mudanças ² para determinar tanto quais testes foram afetados quanto quais mudanças que foram responsáveis pela alteração no comportamento dos testes. O trabalho se assemelha com a abordagem aqui proposta por analisar o impacto que mudanças em cenários de evolução possuem sobre a suíte de testes, mas os objetivos diferem

²Análise que categoriza as mudanças em tipos (Classe adicionada, Método modificado, Variável adicionada, etc.) e às relaciona entre si.

uma vez que essa abordagem busca apresentar quais os fluxos de execução impactados por mudanças que precisam ser re-testados.

MATRIX (SANTELICES et al., 2008) é uma abordagem que analisa o impacto que mudanças entre versões de código possuem sobre a suíte de testes. O trabalho utiliza análise de dependência, execução simbólica parcial e análise dinâmica para (1) aferir a adequação dos testes após as mudanças e (2) gerar novos casos de teste que cubram os comportamentos não testados introduzidos pelas mudanças. Diferente da abordagem aqui proposta, o trabalho exemplifica o objetivo geral dos trabalhos na área de análise de impacto de mudanças aplicada à área de testes de regressão, que é, genericamente, analisar quais testes foram afetados por mudanças e apresentar uma forma de construção, reaproveitamento ou exclusão de casos de teste.

Por fim, existem ainda outras abordagens, inseridas na área de cobertura de código, que se relacionam com o trabalho aqui proposto. Os trabalhos nesta área, diferente dos anteriores, possuem como característica comum, determinar quais partes do código estão sendo executadas pelos testes tendo assim o intuito de encontrar código que não está sendo testado. As abordagens desta área diferem entre si, principalmente, no que diz respeito à escolha da(s) entidade(s) ou critério(s) de cobertura que será(ão) considerada(s) para determinar se o código está sendo testado ou não. Segundo (ALEMERIEN; MAGEL, 2014; LI et al., 2013; SHAHID; IBRAHIM, 2011b), em trabalhos que analisam e resumizam mais de vinte ferramentas de cobertura de código diferentes, apenas três critérios de cobertura ou entidades de código são levados em consideração pela maioria das ferramentas para computar a cobertura do código, que são: cobertura de linha ou bloco de código, cobertura de decisão e cobertura de métodos ou funções.

Assim, tendo em vista a ausência de ferramentas e estudos que analisam os efeitos colaterais das mudanças de código e que buscam determinar a cobertura dos componentes afetados, apresentando como resultado quais os fluxos de execução que foram impactados pelas mudanças e que não estão sendo cobertos por testes, este trabalho objetiva desenvolver uma abordagem que analisa cenários de evolução de código usando análises dinâmicas e estáticas de código considerando tais fluxos. A abordagem busca auxiliar a equipe responsável pelos testes do sistema com uma informação específica e detalhada sobre a execução dos testes que permite melhorar a qualidade da cobertura dos testes da suíte.

1.3 Objetivos

Este trabalho tem como objetivo principal desenvolver uma abordagem que: (i) analisa a cobertura de código levando em consideração os fluxos de chamadas existentes no sistema que foram afetados por evoluções de código, assim como os fluxos de execução oriundos da execução dos testes; e (ii) indica quais fluxos de chamadas do sistema que possuem métodos modificados e não estão sendo cobertos pelos testes atualmente e que, portanto, poderiam ser considerados para melhorar a qualidade dos testes. Dessa forma, baseado no objetivo geral, enumera-se os seguintes objetivos específicos:

- Projetar e implementar uma abordagem que analisa o código de sistemas Java na busca por fluxos de execução não cobertos pelos testes;
- Realizar uma análise comparativa dos principais trabalhos propostos na área de cobertura de testes e ferramentas de análise de impacto de mudanças;
- Determinar a qualidade da suíte do sistema no que diz respeito à cobertura de fluxos de execução que foram afetados por evoluções no código;
- Indicar se houve ou não degradação da suíte de testes levando em consideração a evolução ou não dos fluxos de execução cobertos e não cobertos.

A abordagem proposta utiliza uma combinação entre as análises estática e dinâmica para extrair: (i) os fluxos de execução do sistema; (ii) os fluxos de execução afetados pelas modificações; e (iii) os fluxos de execução que são cobertos pelos testes. Com esses dados obtidos, a abordagem realiza uma segunda análise para determinar quais fluxos não estão sendo cobertos pelos testes.

A ferramenta concebida na abordagem foi desenvolvida como um framework orientado a objetos utilizando a linguagem de programação Java. Como ferramentas de suporte, o WALA³ foi utilizado na análise estática e o AspectJ⁴ proveu o suporte na análise dinâmica, juntamente com a utilização do banco de dados PostgreSQL⁵.

³<http://wala.sourceforge.net/wiki/index.php/MainPage>

⁴<https://eclipse.org/aspectj/>

⁵<http://www.postgresql.org/>

1.4 Organização do trabalho

Além deste capítulo introdutório, este documento apresenta outros quatro capítulos que estão organizados como segue:

- O Capítulo 2 apresenta os conceitos que apoiam a fundamentação teórica do trabalho;
- O Capítulo 3 apresenta uma visão geral da abordagem;
- O Capítulo 4 apresenta a avaliação da abordagem proposta, no qual os resultados são coletados e analisados.
- O Capítulo 5 apresenta os trabalhos relacionados com a proposta.
- O Capítulo 6 apresenta as conclusões e os trabalhos futuros.

2 Fundamentação teórica

Neste capítulo, abordamos os fundamentos teóricos sobre os quais este trabalho foi desenvolvido. Inicialmente, apresentamos os conceitos básicos de análise de impacto de mudanças. Discutimos sobre as diferenças entre as abordagens estática e dinâmica e ponderamos as vantagens e desvantagens de cada uma (Seção 2.1). Em seguida o conceito de cobertura de código e a definição de seus critérios é apresentada na Seção 5.1.

2.1 Análise do Impacto de Mudanças

A manutenção de software é a fase mais custosa e complicada dentro do ciclo de vida do desenvolvimento (REN, 2007). A evolução dos sistemas ocorrem por diversas razões: correção de falhas, adição de novas funcionalidades, refatoração para melhorar a performance, adaptação de código para novas versões de bibliotecas, etc, mudanças são inevitáveis. Atualmente, os sistemas estão se tornando cada vez maiores, com muitos componentes que interagem entre si, o que dificulta cada vez mais a atividade de manutenção, onde uma pequena mudança pode acarretar diversas falhas por todo o sistema.

Segundo Bohner, uma mudança no código do sistema compreende: 1) entender como a mudança afeta o software; 2) implementar a mudança proposta; e 3) testar o sistema alterado (BOHNER, 2002). Tipicamente, a primeira etapa envolve especificar a mudança e avaliar como ela afetará o software, se for implementada. A atividade de avaliar como a mudança afeta o software é denominada de *análise de impacto de mudanças*.

Embora, originalmente, tenha sido criada para analisar mudanças antes de sua implementação, a análise de impacto é empregada em diferentes momentos no processo de manutenção e com diferentes finalidades. Quando realizada antes de a mudança ocorrer, pode facilitar algumas atividades dentro do processo de desenvolvimento. Nesse contexto, a análise de impacto pode auxiliar a estimativa de custo e tempo necessários para realizar mudanças. Quando a análise de impacto é realizada após a implementação da mudança,

ela ajuda no processo de testes de regressão, identificando os testes que necessitam ser re-executados para verificar a integridade do software ou provendo informações que possam ajudar na melhoria dos testes. Neste trabalho foi adotado a atividade de análise de impacto realizada após a implementação da mudança.

Um fator crítico da análise de impacto diz respeito ao encadeamento de mudanças. A modificação em uma unidade do código fonte geralmente impacta estruturas dependentes, podendo introduzir acidentalmente defeitos. Um problema típico é a dificuldade de identificar essas estruturas, principalmente em sistemas com alto acoplamento e baixa coesão. A análise de impacto, que inclui analisar as dependências entre estruturas, pode ser utilizada para definir o escopo e a complexidade de determinada modificação, bem como os riscos associados.

Um impacto de uma mudança representa uma parte do software que é determinantemente afetada e que merece investigação. Essa inspeção tem como objetivo manter a estabilidade do software, o que representa a sua resistência a um potencial efeito colateral que um programa pode ter quando modificado. Esse efeito colateral, por sua vez, é considerado como um erro ou outro comportamento inesperado resultante de uma modificação em um lugar no código que afeta outras partes relacionadas.

2.1.1 Estratégias para Análise de Impacto

Existem diversas estratégias para analisar o impacto de determinada mudança no software, com diferentes níveis de automação. Arnold e Bohner apresentam algumas dessas estratégias (ARNOLD; BOHNER, 1993):

- navegação pelo código, abrindo e fechando arquivos relacionados;
- pesquisa por especificações e documentação para determinar o escopo da mudança;
- análise de rastreabilidade para identificar artefatos a serem modificados;
- análise de dependência pelo particionamento do sistema para verificar o subconjunto desse sistema que pode ser afetado pelas modificações.

Observa-se que estas estratégias podem ser manuais e automatizadas por algoritmos. As estratégias manuais geralmente são desempenhadas por humanos e não necessitam de uma infraestrutura específica, enquanto as automatizadas necessitam dessa infraestrutura para rastrear objetos e estimar o impacto de uma mudança.

Estratégias de análise de impacto automatizadas geralmente empregam algoritmos para identificar a propagação de mudanças e seus impactos indiretos. Para tanto, possuem como pré-requisito os grafos de relacionamento, que constituem uma informação semântica que evidencia as associações entre entidades computacionais. Estratégias como análise de rastreabilidade e dependência são geralmente utilizadas para identificar os impactos de mudanças.

Análise de Rastreabilidade se refere a habilidade de definir e recuperar associações entre entidades, incluindo requisitos ou outros tipos de artefatos de software. Um ponto fundamental para rastreabilidade consiste em perceber um sistema como um conjunto de documentos e artefatos que descrevem o software em diferentes níveis de abstração e não apenas como uma implementação.

Análise de Dependência, foco deste trabalho, envolve o exame detalhado das relações de dependência entre as entidades do programa. Permite uma avaliação de dependências de baixo nível, ou seja, refere-se a análise do código fonte.

As técnicas de análise de dependência incluem, dentre outras, a análise de fluxo de dados (*data flow analysis*), a análise de fluxo de controle (*control flow analysis*), análise baseada nos grafos de chamadas e fatiamento de programa (*program slicing*). Um grafo de chamadas é um grafo direto no qual os nodos representam métodos e uma aresta transitiva entre A e B significa que A pode chamar B . Através da definição de dependência direta entre os métodos, é possível avaliar o impacto através da manipulação de um nó específico. Neste trabalho, as análises são baseadas na construção de grafos de chamadas a nível de método e na técnica de *backward slicing*.

Não podemos indicar vantagens e desvantagens entre as duas técnicas pois elas não são concorrentes, o que pode ser afirmado é que existem casos em que uma é mais adequada e indicada do que a outra. Se o problema abordado tem interesse no fluxo de execução que antecede ou precede as modificações no código.

As abordagens e técnicas que utilizam a análise de dependência, podem ser divididas em duas categorias: análise de impacto estática ou dinâmica. As técnicas estáticas analisam o código-fonte dos sistemas, enquanto as técnicas dinâmicas analisam os rastros ou fluxos (traces) de execução do sistema. Mais detalhes sobre as duas técnicas serão apresentadas a seguir.

2.1.2 Análise de Impacto Estática

As técnicas estáticas analisam o código-fonte do software para identificar os elementos possivelmente impactados por uma mudança. Elas normalmente mapeiam o sistema para um conjunto de grafos, no qual os nós representam as entidades e as arestas são as dependências entre as entidades. Esses grafos podem ser construídos em nível de método ou em um nível mais detalhado, o de sentenças (RYDER; TIP, 2001).

Análises estáticas em nível de sentenças usam o fatiamento de programas para segmentar o sistema em grafos de fluxo de controle e dados. Elas são consideradas de granularidade fina, pois realizam uma análise detalhada e o resultado inclui todas as dependências de uma mudança em nível de sentença. Essa é uma abordagem que gera um grande número de falso-positivos, pois considera todas as possíveis entradas e todos os possíveis comportamentos do sistema.

Já as análises estáticas em nível de método, usam a técnica de fechamento transitivo em grafos de chamadas, ou técnicas semelhantes para identificar as chamadas possivelmente impactadas por uma mudança. São consideradas de granularidade grossa, pois analisam o software com menor nível de detalhe.

Existem duas técnicas de fechamento transitivo, na primeira, *forward slicing*, assume-se que uma mudança em um procedimento p de um programa P pode impactar, potencialmente, qualquer nó alcançável a partir de p no grafo de chamadas G de P . O fechamento transitivo de G calcula as relações de possíveis impactos entre todos os procedimentos de P alcançáveis a partir de p . A segunda técnica, *backward slicing*, possui o conceito contrário, assume-se que uma mudança realizada em p pode impactar qualquer nó que alcança ou acessa p em G de P (RYDER; TIP, 2001).

2.1.3 Análise de Impacto Dinâmica

As técnicas dinâmicas analisam fluxos de execução de programas para identificar os elementos impactados por uma mudança. Há duas categorias de análise dinâmica: *online* e *offline*. Elas se assemelham em relação aos dados coletados, à forma de analisar os impactos de uma mudança e aos resultados encontrados. A diferença é que a *offline* coleta os dados dos rastros para analisá-los após o fim da execução, enquanto a *online* analisa os rastros durante a execução do programa. Essa diferença entre as duas análises determina a velocidade de obtenção dos resultados e sua confiabilidade de corretude, ou seja, enquanto a análise *online* consegue obter um resultados mais rapidamente, a análise

offline provê uma maior confiabilidade uma vez que possui os dados como um todo para fazer uma análise mais abrangente.

A análise de impacto dinâmica computa, aproximadamente, o conjunto dinâmico de elementos impactados, pois ela depende do conjunto de execuções e da entrada para cada execução. Como não é possível assegurar que a análise dinâmica capture rastros para todas as possíveis execuções de um programa e todos os possíveis valores de entrada, sabe-se que técnicas dinâmicas geram falso-negativos (LAW; ROTHERMEL, 2003; CAI; SANTELICES, 2014).

2.2 Cobertura de código

O teste de software é uma etapa no desenvolvimento de software que se tornou imprescindível para que a equipe possa garantir um mínimo de confiança, segurança e qualidade do sistema que está sendo desenvolvido (MYERS; SANDLER; BADGETT, 2011). O ato de prover uma suíte de testes e colocar o código a prova, pode mostrar a presença de falhas (*bugs*), mas nunca a sua ausência (DAHL; DIJKSTRA; HOARE, 1972). Geralmente, os testes são usados de duas formas: na primeira os testes são empregados como forma de suporte na fase de desenvolvimento, provendo informações sobre inconsistências no código. Caso algo de inconsistente seja encontrado, medidas reparadoras são projetadas, implementadas e aplicadas; na segunda, os testes são usados como forma de certificação de confiança, ou seja, os testes são realizados com o intuito de aferir se o software realiza as funções, para que ele foi planejado e desenvolvido, de maneira adequada (BEIZER, 1990).

De acordo com Beizer, o ato de testar é simples, tudo que é necessário é encontrar um fluxo de execução (grafo) e prover testes que executem esse fluxo de acordo com critérios pré-definidos. O que seria então um critério de teste? Os critérios de teste definem o que constitui um teste adequado, ou seja, quais propriedades do software precisam ser exercitadas (executadas em diversas formas diferentes) para que o teste seja completado com sucesso, isto é, que os testes não detectem erros de execução (ZHU; HALL; MAY, 1997).

Alguns termos usados na área são "*exhaustive testing*", que é o ato de prover um grande número, mas limitado, de entradas para testar o código múltiplas vezes e em cenários diversos; e "*full coverage*", que pode ser traduzido para um ambiente em que os testes abrangem toda a aplicação em diversas condições e cenários de execução diferentes. Todavia, a definição para estes termos possui uma limitação de software teórica fundamental, onde o número de entradas para a maioria dos programas é tão grande que podem ser

considerados um valor infinito que não pode ser explicitamente enumerado. O conceito de critérios de teste formais veio para suprir esse problema, uma vez que não é possível desenvolver testes com todas as entradas possíveis, os critérios de cobertura são usados para definir quais entradas serão usadas (AMMANN; OFFUTT, 2008).

Durante a etapa de testes, diferentes estratégias podem ser adotadas de acordo com os aspectos de qualidade que serão avaliados. O desenvolvimento da suíte de testes requer um planejamento cuidadoso e um processo que determine com precisão as atividades necessárias para o desenvolvimento adequado dos testes (MCGREGOR; SYKES, 2001). Tradicionalmente, as equipes de desenvolvimento adotam o modelo V de testes (FEWSTER; GRAHAM, 1999; AMMANN; OFFUTT, 2008), o qual é apresentado na Figura 1 a seguir.

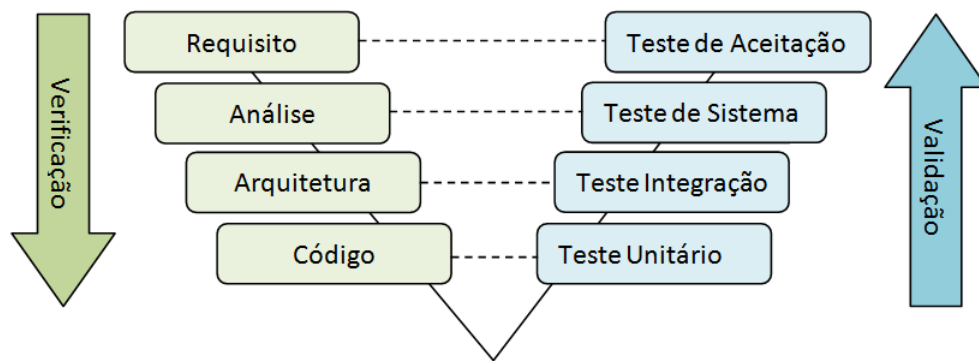


Figura 1: Modelo V em testes de software

Como esta dissertação analisa o código das aplicações, o escopo está limitado ao domínio dos testes unitários, que diz respeito ao teste de unidades ou processos de um sistema, que é representado pelas funções no C e C++, sub rotinas no Fortran e métodos no Java, por exemplo (AMMANN; OFFUTT, 2008).

2.3 Critérios de cobertura

A cobertura de código é tida como uma técnica de testes caixa branca que mede o quanto do programa é executado pela suíte de testes. Cada teste na suíte usa asserções para especificar o comportamento esperado do programa. A execução da suíte de testes tenta verificar se o programa está funcionando como o esperado. Cada execução computa o nível em que o código se encontra coberto pelos testes levando em consideração um critério específico, que é definido em termos de unidades de medida, como métodos ou linhas de código. Nesta seção, uma visão geral dos critérios de cobertura será apresentada.

2.3.0.1 Cobertura de Segmento

Muitos desenvolvedores e engenheiros de testes recorrem ao uso da cobertura de segmento ou de linha de código (*statement coverage*) por causa da sua simplicidade de implementação e disponibilidade de ferramentas que dão suporte à este tipo de métrica (BEIZER, 1990; ZHU; HALL; MAY, 1997).

Um segmento ou *statement*, é a entidade no contexto de linguagens de programação, que tipicamente corresponde a menor e indivisível unidade de execução, em outras palavras, uma linha de código que contenha uma afirmação.

O objetivo da cobertura de segmento é informar se cada segmento do sistema está coberto por ao menos um caso de teste, ou seja, se cada linha de código foi executada pelos testes. O benefício de se utilizar tal critério é a possibilidade de identificar quais blocos ou linhas de código não foram executadas pelos testes.

De todos os critérios de cobertura, este é o menos poderoso em identificar falhas e no que diz respeito ao nível de abrangência, pois não considera estruturas de controle, apenas linhas e blocos código, portanto falhas simples que poderiam estar contidas nessas estruturas não serão enxergadas pela abordagem que utiliza apenas este critério de cobertura.

2.3.0.2 Cobertura de Decisão

Nenhuma aplicação pode ser desenvolvida em um modo contínuo de codificação, em algum momento é preciso ramificar o código para que funcionalidades específicas sejam executadas. Ramificar o código quer dizer que estruturas de decisão são inseridas e dependendo das condições aferidas, partes do código diferentes serão executadas. Um exemplo de ramificação pode ser visto na Figura 2

Um ramo nada mais é que a saída de uma estrutura de decisão, então a cobertura de decisão ou *branch coverage*, busca identificar quais ramos foram testados, ou seja, para cada estrutura de decisão no código (*if*, *switch*, *while*, etc.), toda saída possível deve ser testada ao menos uma vez. Mesmo sendo mais robusta que a cobertura de segmento, a cobertura de decisão possui a desvantagem de ignorar ramos com expressões booleanas complexas (AMMANN; OFFUTT, 2008; ZHU; HALL; MAY, 1997).

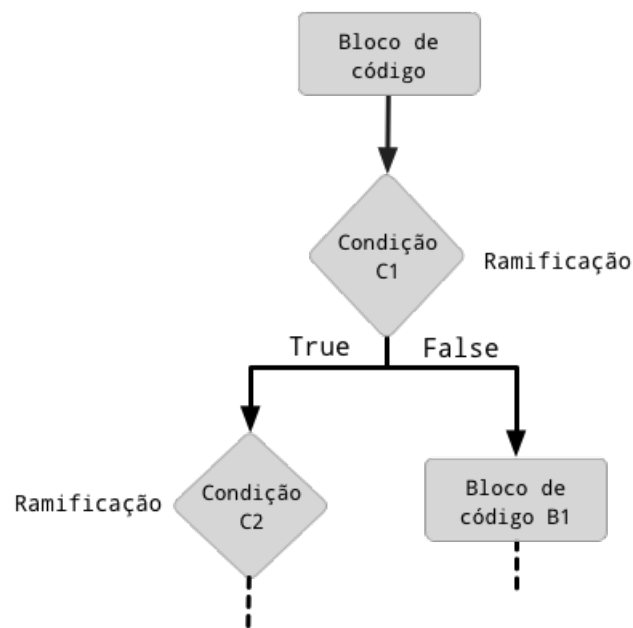


Figura 2: Exemplo de Ramificação

2.3.0.3 Cobertura de Fluxo

A cobertura de fluxo ou *path coverage*, diz respeito ao fluxo de execução desde o início até o fim de um método, portanto um método com N ramificações, possui 2^N fluxos ou caminhos possíveis. Um caminho é uma sequência única de ramificações, desde a entrada até a saída do método. Um exemplo de fluxos de métodos pode ser visto na Figura 3.

Como a cobertura de decisão, a cobertura de fluxo também busca identificar quais ramos foram testados, a diferença é que os ramos são tratados independentes uns dos outros, ou seja, cada ramo é executado sem alterar as condições dos ramos anteriores, o que permite uma maior flexibilidade e robustez para os testes (BEIZER, 1990; ZHU; HALL; MAY, 1997).

Caso o método possua algum *loop* de iteração, dependendo das entradas que são passadas pelos testes, o número de caminhos possíveis pode chegar ao infinito, para isso, variações deste critério foram desenvolvidos que levam em consideração, por exemplo, a complexidade ciclomática para reduzir o número de caminhos que precisam ser testados. A definição de complexidade ciclomática e uma descrição dos critérios derivados fogem do escopo desta dissertação.

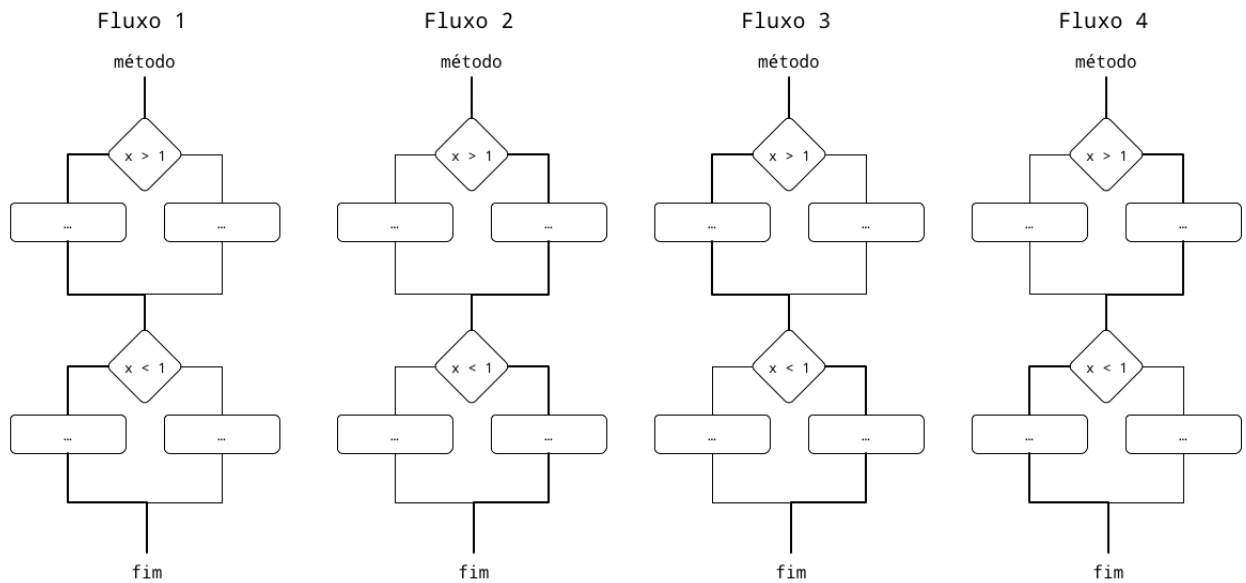


Figura 3: Fluxos de execução

2.3.0.4 Cobertura de Fluxo de Chamadas Entre Métodos

Diferente da cobertura de fluxo, a cobertura de chamada ou *call coverage*, trata especificamente do fluxo de chamada entre os métodos da aplicação, ou seja, as estruturas de condição e repetição não são levadas em consideração, apenas as chamadas diretas entre os métodos são capturadas. Diferente das demais, este critério tem o objetivo de informar quais métodos foram ou não executados dentro dos testes, nada sobre o conteúdo do método ou a quantidade de chamadas é medida, apenas se o método foi invocado em algum momento do teste ou não.

Por medir o fluxo de chamadas entre métodos, esta métrica, dependendo do tipo de análise em que estiver inserida, pode ser formalmente relacionada com os seguintes termos definidos por (IEEE, 1990): (i) *Call graph* - Um diagrama que identifica os módulos em um sistema ou programa de computador e mostra quais módulos chamam uns aos outros; (ii) *Execution trace* - Um registro da sequência de instruções executadas durante a execução de um programa de computador. Muitas vezes na forma de uma lista de *code labels* (identificador de código) encontrados enquanto o programa é executado; (iii) *Path analysis* - Análise de um programa de computador para identificar todos os caminhos ou fluxos (*paths*) possíveis, seja para detectar fluxos incompletos ou para descobrir porções do programa que não estão em nenhum fluxo.

3 Uma abordagem para análise de cobertura de código em cenários de evolução

Este capítulo apresenta uma visão geral da abordagem que foi proposta nesta dissertação. A abordagem se insere dentro do contexto de evolução de sistemas de software e tem como principal objetivo analisar quais fluxos de execução foram afetados pelas mudanças e informar quais fluxos que não estão sendo testados pela suíte de testes. Para isso, este trabalho busca desenvolver uma abordagem que: (i) analisa a cobertura de código levando em consideração os fluxos de chamadas existentes no sistema que foram afetados pela evolução do código, assim como os fluxos de execução oriundos da execução dos testes; e (ii) indica quais fluxos de chamadas do sistema não estão sendo cobertos pelos testes atualmente e que, portanto, poderiam ser considerados para melhorar a qualidade dos testes.

3.1 Visão Geral da Abordagem

A abordagem está organizada em quatro componentes principais: (i) o componente de construção do grafo de chamadas da aplicação (análise estática); (ii) o componente de construção do grafo de chamadas dos testes (análise dinâmica); (iii) o componente de análise e extração das evoluções; (iv) e o componente de análise dos grafos e obtenção dos resultados. As subseções seguintes detalham cada componente da abordagem.

Componente de construção do fluxo de chamadas da aplicação

A primeira atividade da abordagem, ilustrada na Figura 4 consiste em obter os fluxos de execução do sistema na forma de um grafo. O grafo é constituído de nós que representam os métodos chamadores (*caller*) e chamados (*callee*). Assim percorrendo cada caminho do grafo até o último retorno da cadeia de chamadas, temos os fluxos de execução dos métodos

da aplicação, que é denominado de hierarquia de chamada (*call hierarchy*) e será utilizada como base de dados para a futura busca por caminhos afetados pelas evoluções.

Para obter a hierarquia de chamadas da aplicação, a abordagem utiliza o conjunto de bibliotecas de análise estática de código WALA (T.J. Watson Libraries for Analysis). Tal solução foi escolhida por sua conhecida robustez e vasta utilização na área de análise de código. O WALA provê várias técnicas e algoritmos de análise de código, mas para o escopo dessa abordagem, a técnica de análise de pontos (*pointer analysis*) juntamente com uma estrutura de grafo de propagação e o algoritmo de construção de grafos Zero One CFA foram escolhidas por proporcionarem um balanceamento ideal de precisão, robustez, consumo de tempo de execução e espaço para armazenamento (GROVE; CHAMBERS, 2001; HIND; PIOLI, 2000).

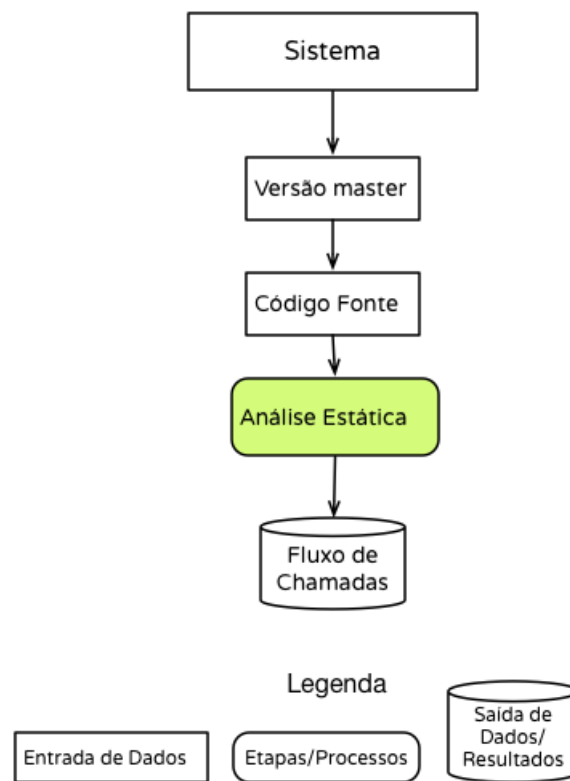


Figura 4: Etapa de construção do grafo de chamadas da aplicação

Componente de construção do fluxo de chamadas dos testes

Após a obtenção da hierarquia de chamadas da aplicação usando análise estática, a segunda atividade da abordagem é capturar as chamadas que a suíte de testes da aplicação exercita, ilustrada na Figura 5, através de uma análise dinâmica em tempo de execução. Para tal atividade, a linguagem orientada a aspectos AspectJ foi utilizada com o objetivo de instrumentar o código Java sendo executado. A instrumentação com AspectJ é feita a

partir da anotação do código de testes provida pelo framework de testes JUnit, ou seja, todos os testes que continham a anotação de teste *@Test* foram analisados.

O fluxo de chamadas oriundo da execução dos testes tem como objetivo prover a base de dados onde será analisado quais fluxos de chamadas estão cobertos pelos testes e quais não estão. Diferente da etapa anterior, neste passo, apenas as chamadas feitas pela suíte de testes são de interesse para a abordagem.

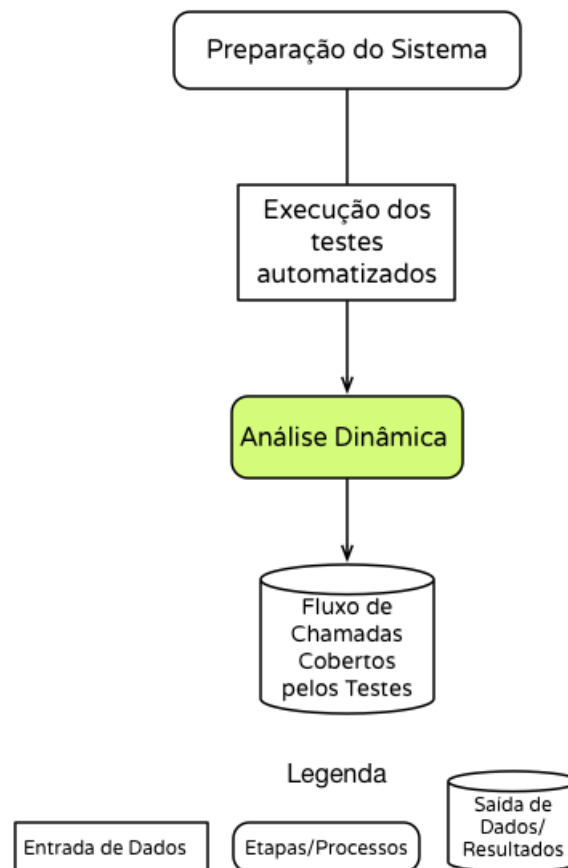


Figura 5: Etapa de construção do fluxo de chamadas dos testes

Componente de análise e extração das evoluções

O cenário para o qual a abordagem foi projetado é o de sistemas que estão em constante evolução, para que assim os resultados obtidos pela abordagem possam ser de utilidade no dinâmico ambiente de evolução, provendo dados que ofereçam suporte para uma visão mais profunda de como está o nível de cobertura do código em desenvolvimento. Assim, como pode ser visto na Figura 6, a partir das mudanças contidas nas iterações de evolução, a abordagem extrai quais métodos foram modificados para prover o dado de entrada para a próxima etapa de obtenção dos resultados. A extração das evoluções é feita a partir da análise dos *commits*, providos pelo sistema de controle de versão distribuído Git, onde é feito uma análise do que mudou em relação a versão anterior que está sendo

analisada.

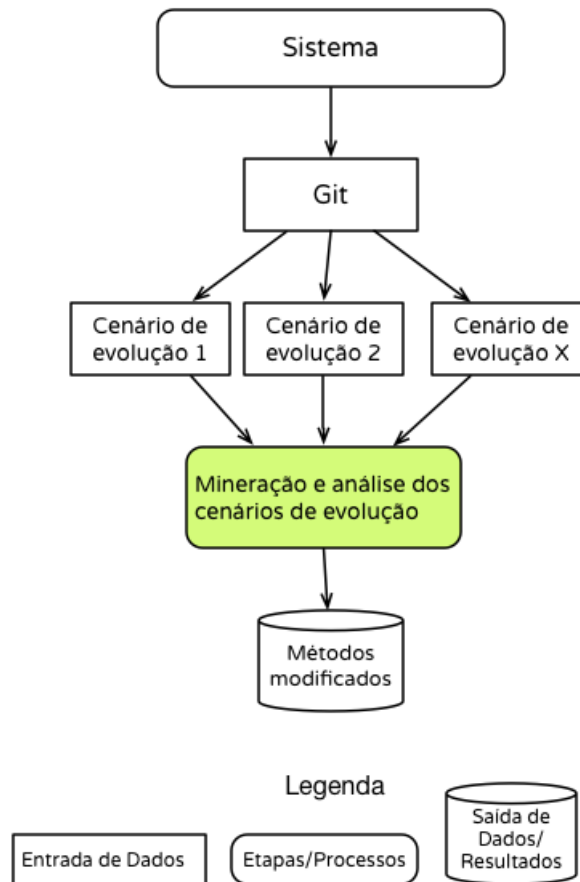


Figura 6: Etapa de análise e extração das evoluções

Componente de análise dos fluxos e obtenção dos resultados

A última etapa da abordagem, ilustrada na Figura 7, consiste em analisar os fluxos de execução obtidos na análise estática na busca por fluxos que contenham os métodos extraídos das evoluções, ou seja, os fluxos que foram afetados pelas mudanças. A abordagem então analisa se esses fluxos afetados já estão sendo executados pelos testes a partir de uma segunda análise feita usando a base de dados que foi obtida pela análise dinâmica.

Com a informação de quais fluxos de execução estão ou não sendo executados pelos testes, a abordagem finaliza a execução provendo os seguintes dados: (i) fluxos de chamada da aplicação; (ii) fluxos afetados pelas evoluções; (iii) dentre os fluxos afetados, quais estão sendo executados pelos testes; (iv) dentre os fluxos afetados, quais não estão sendo executados pelos testes; (v) um percentual da cobertura dos testes levando em consideração a quantidade de fluxos afetados.

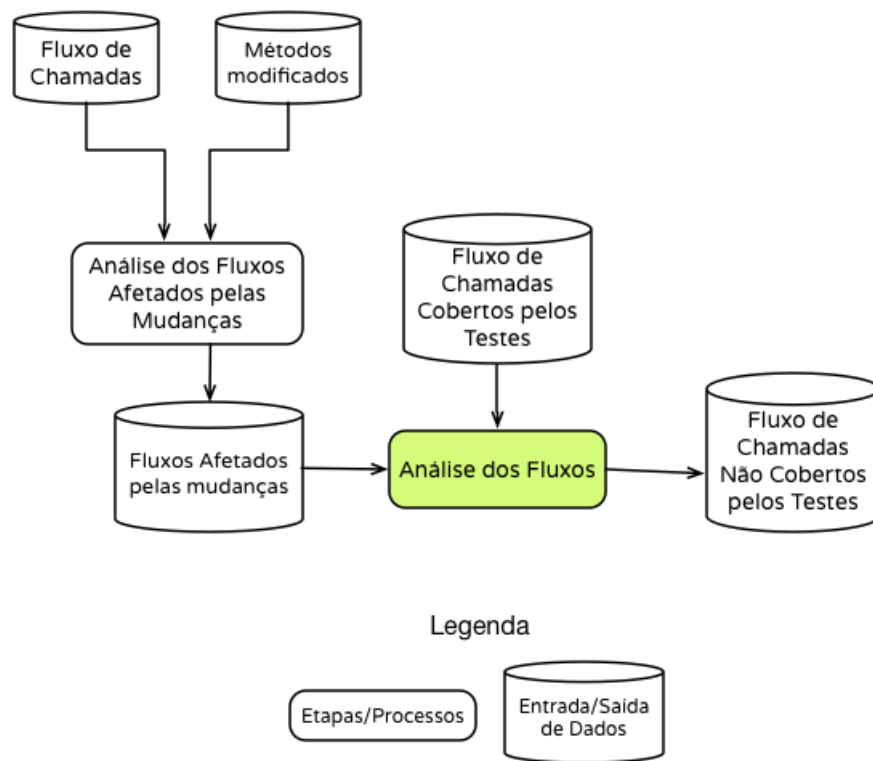


Figura 7: Etapa de análise dos fluxos e obtenção dos resultados

3.2 Arquitetura da Ferramenta

A ferramenta desenvolvida para oferecer suporte a abordagem proposta foi projetada para ser usada dentro do ambiente integrado de desenvolvimento Eclipse. A ferramenta se comporta, até o momento, como um projeto Java que contém arquivos de propriedades que devem ser editados com informações do código e respectivos *releases* a serem analisados.

A figura 8 apresenta a estrutura geral da arquitetura de implementação da ferramenta. O pacote *main* possui apenas a classe *Main* que é responsável pela execução de toda a abordagem. Em seguida, temos o pacote *dynamic*, que tem a função de transformar as entradas capturadas pela análise dinâmica em fluxos de chamadas, ou seja, a classe *DynamicFlowExport* recebe as tabelas do banco de dados relacionadas entre si e gera os fluxos de execução para o formato desejado. Para a conexão com a base de dados, existem dois pacotes, o *model* e o *database*, neles estão contidas as classes que dão suporte à conexão com o banco de dados PostgreSQL. A conexão com o banco de dados foi implementada de forma que não há acoplamento com o banco de dados provedor, ou seja, caso necessário a solução de armazenamento dos dados pode ser substituída.

O pacote *wala* contém a classe *CallGraphWALA* que é responsável pela análise estática da abordagem, ou seja, a construção do grafo de chamadas da aplicação. Em seguida temos

o pacote *graph_analysis* que desempenha o papel de comparação dos grafos e obtenção dos resultados, executado pela classe *GraphAnalysis*.

Em *connectors* a função de conexão com os sistemas de versionamento (Git) e hospedagem (Github) do código são realizadas. Esse módulo também foi implementado visando uma possível substituição ou uso em conjunto de soluções diferentes, portanto o pacote *connectors* provê as classes abstratas *EvolutionConnector*, que possui as informações e métodos acerca do sistema de hospedagem e/ou gerenciamento das evoluções, e *SCMConnector*, que possui as informações e métodos do sistema de versionamento de onde o código será baixado ou lido. Na abordagem, a solução de hospedagem escolhida foi o Github, portanto o código de conexão com esse serviço se encontra na classe *GithubConnector* e a solução de onde o código será extraído escolhida foi o Git, tendo seu código de conexão na classe *GitConnector*.

O pacote *jdt* contém o código responsável pela identificação de quais métodos que foram modificados dentre as evoluções analisadas. Por não haver uma solução para esse problema nos serviços de gerenciamento das evoluções, optou-se por implementar uma solução própria que utiliza as bibliotecas do Eclipse Java Development Tools (JDT). Por último, o pacote *util* contém classes de suporte à abordagem, a classe *MemberUtil* contém código relacionado a formatação e a classe *PropertiesUtil* faz a conexão com os arquivos de propriedade da abordagem.

3.3 Demonstração da Abordagem em um Ambiente Controlado

Com o intuito de validar os resultados obtidos e possuir um maior controle sobre as mudanças ocorridas nas evoluções, um sistema foi desenvolvido para simular os cenários de evoluções e possibilitar uma avaliação controlada da abordagem. O intuito desta análise feita em um ambiente controlado é indicar que a ferramenta consegue detectar os fluxos de execução esperados tendo em vista mudanças realizadas propositalmente em lugares específicos.

A aplicação desenvolvida contém 7 arquivos (4 classes de código e mais 3 para execução dos testes) e 92 linhas de código. O código foi desenvolvido tendo em mente que as classes seriam dependentes umas das outras, para que quando houvesse uma mudança em determinado método, essa mudança afetasse diversos fluxos de chamadas. Para aferir o estado de cobertura dos fluxos para alguns métodos, intencionalmente, os seus testes não

foram desenvolvidos.

A Figura 9 apresenta o fluxo de chamadas do sistema simulado. Para a análise, os 2 cenários de evolução seguintes foram analisados.

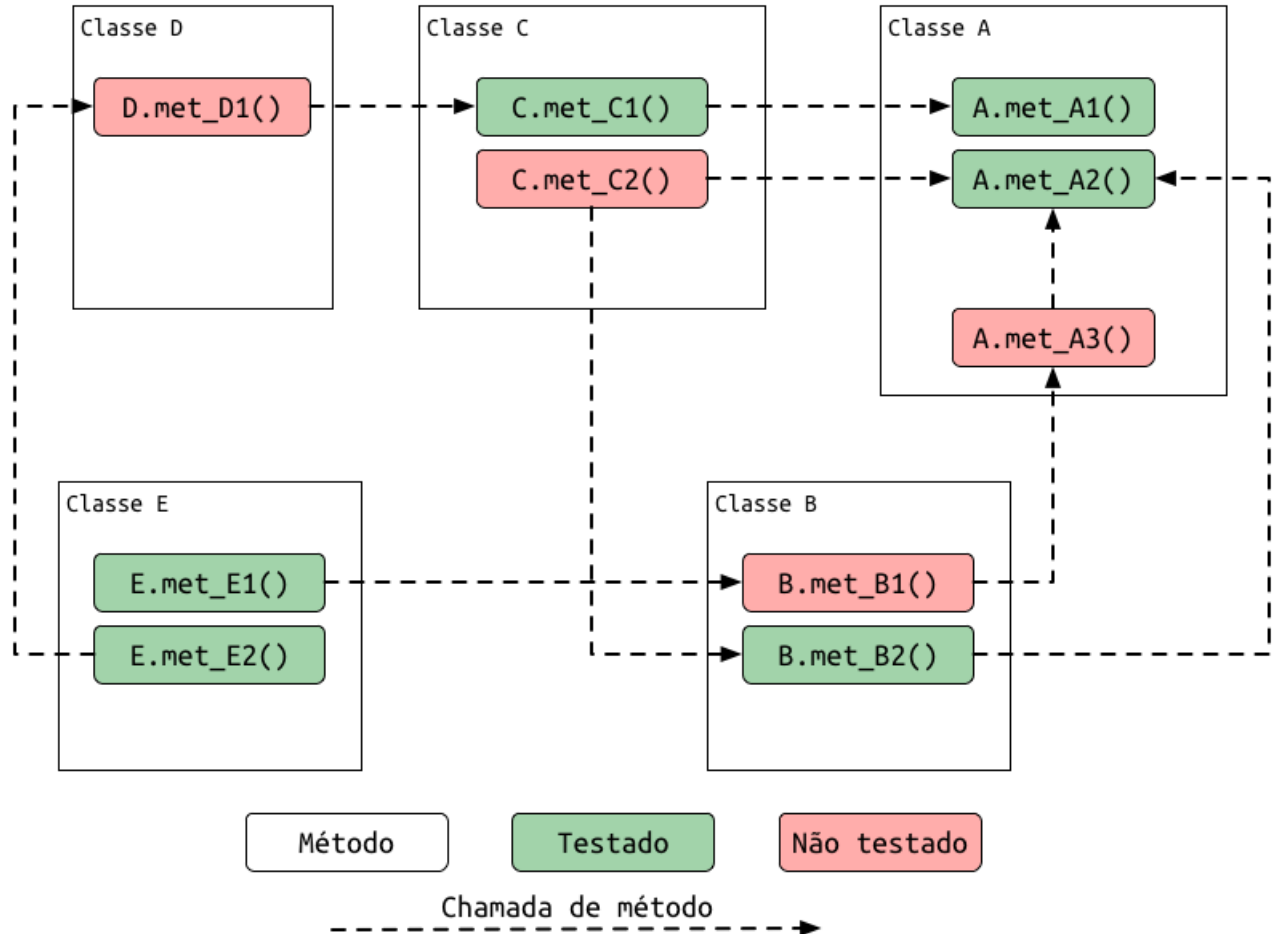


Figura 8: Fluxo de Chamadas do Sistema Simulado

Cenário de evolução 1

Para o primeiro cenário de evolução, uma mudança no método `A.met_A1()` foi realizada. A Figura 10 apresenta quais fluxos de execução foram afetados pela mudança realizada.

Como resultado, a abordagem identificou que apenas o fluxo $D.met_D1() > C.met_C1() > A.met_A1()$ deve ser re-testado pois não há testes que executam diretamente o método `D.met_D1()`.

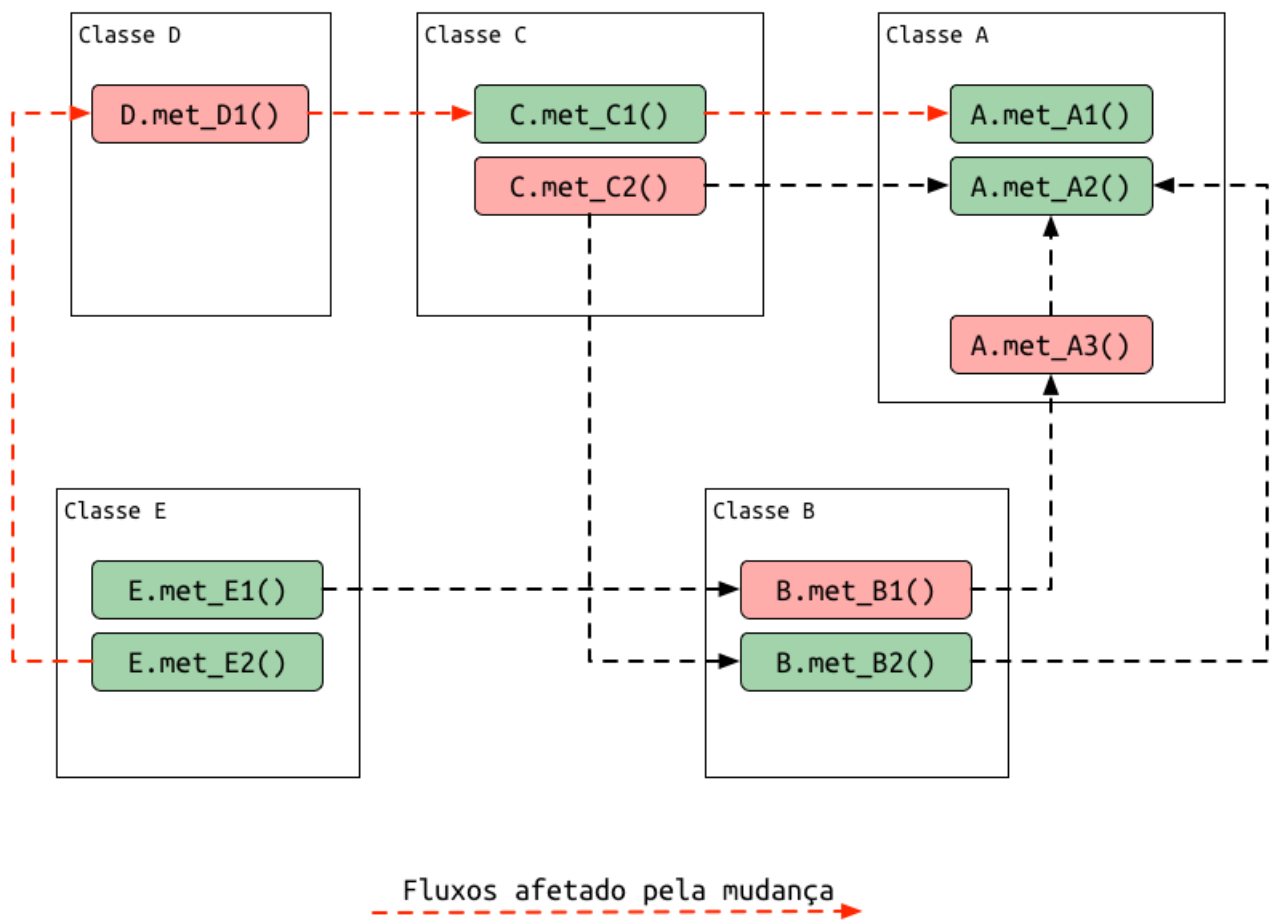


Figura 9: Fluxos de Execução Afetados pela Mudança no Cenário 1

Cenário de evolução 2

Para o segundo cenário de evolução, a mudança foi realizada no método A.met_A2(), os fluxos de execução afetados pela mudança são apresentados na Figura 11.

A abordagem identificou que 4 fluxos de execução precisam ser re-testados, são eles:

1. B.met_B1(int,int) > A.met_A3() > A.met_A2()
2. A.met_A3() > A.met_A2()
3. C.met_C2() > A.met_A2()
4. C.met_C2() > B.met_B2() > A.met_A2()

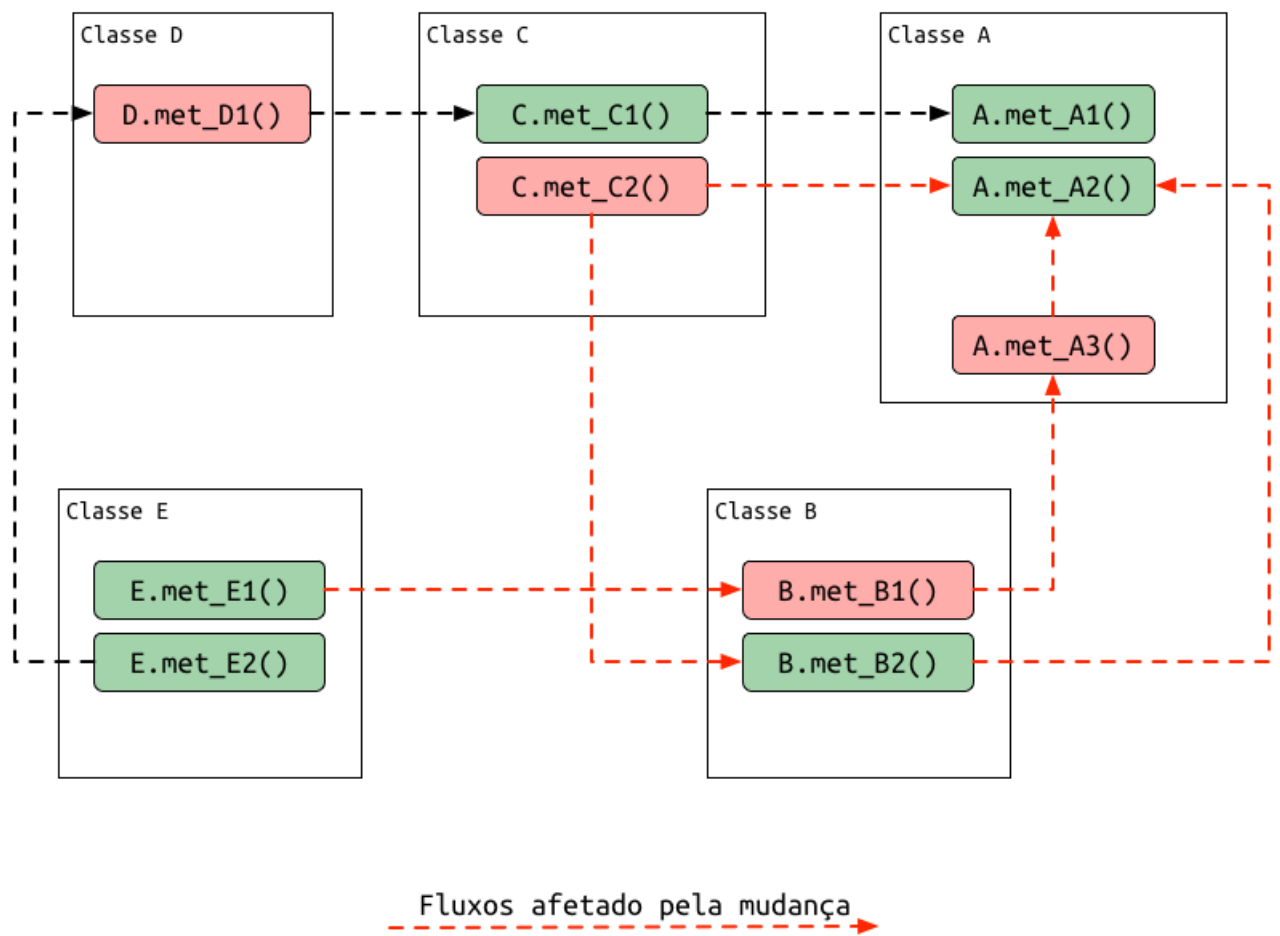


Figura 10: Fluxos de Execução Afetados pela Mudança no Cenário 2

4 Avaliação da Abordagem Proposta

Este capítulo apresenta a avaliação da ferramenta de análise de impacto de mudanças desenvolvida nessa dissertação. O principal objetivo do estudo de avaliação é analisar as evoluções dos sistemas selecionados, e indicar, se presente, quais fluxos de execução, afetados por mudanças, não estão sendo executados pelos testes automatizados.

O restante deste capítulo está organizado da seguinte forma: A seção 4.1 e suas subseções apresentam a configuração do estudo; Na seção 4.2 os resultados do estudo são apresentados e discutidos.

4.1 Configuração do Estudo

Esta seção apresenta informações sobre a configuração do estudo, tais como, os objetivos e questões de pesquisa (Subseção 4.1.1), as fases do estudo (Subseção 4.1.2), uma descrição dos sistemas analisados (Subseção 4.1.3) e as métricas adotadas na avaliação (Subseção 4.1.4).

4.1.1 Objetivos e Questões de Pesquisa

O principal objetivo do estudo foi identificar quais fluxos de execução afetados pelas evoluções não estão sendo testados. Para tanto, este estudo foi realizado com o intuito de responder as seguintes questões de pesquisa:

- **RQ1.** Qual a porcentagem de fluxos de execução afetados pelas evoluções que não são cobertos por suítes de testes automatizadas dentro dos projetos *open-source* analisados?
- **RQ2.** Considerando o histórico de evolução dos projetos, existe degradação ou melhoria da suíte de teste no que se refere a cobertura de fluxos modificados e não cobertos?

A primeira questão de pesquisa busca analisar a qualidade das suítes de testes dos sistemas analisados sob a perspectiva da quantidade de fluxos que sofreram modificações e dentre estes fluxos, quais já se encontram cobertos pelos testes e quais necessitam ser re-testados.

A segunda questão diz respeito ao histórico de evolução da suíte de testes a partir dos resultados obtidos da primeira questão, ou seja, com a quantidade de fluxos de execução afetados e não testados, é possível aferir se houve uma melhoria ou degradação da suíte de testes ao longo do tempo.

4.1.2 Fases do estudo

A execução do estudo foi dividida em três fases, conforme é apresentado a seguir: (i) seleção dos sistemas e evoluções a serem analisados; (ii) execução da mineração de mudanças e das análises estática e dinâmica; (iii) análise e interpretação dos resultados.

4.1.2.1 Fase 1 - Seleção dos Sistemas e Evoluções

O primeiro passo na realização do estudo de avaliação foi a seleção dos projetos de aplicações Java a serem analisados pela ferramenta. Tal seleção seguiu os seguintes critérios: (i) o repositório deveria ser de código aberto e o sistema desenvolvido na linguagem de programação Java; (ii) o repositório deveria ter suas mudanças gerenciadas pelo sistema de versionamento de código Git; (iii) o repositório deveria ter uma suíte de testes JUnit; (iv) o código do sistema e evoluções selecionadas não deveriam conter erros de compilação e falhas na execução dos testes.

Assim, levando em consideração os critérios descritos acima, os seguintes projetos foram selecionados, para a realização desse estudo de avaliação: Bukkit¹, CoolReader², Google Auth Librabry³, GUJ⁴, Mamute⁵ e Vraptor⁶.

A seleção das evoluções a serem analisadas diz respeito a seleção de quais *commits* serão minerados para identificação dos métodos modificados. A ferramenta foi desenvolvida de forma a permitir duas opções de análise: (i) análise de *commit* individualmente, onde é possível fazer a mineração de apenas um *commit* específico; (ii) análise do intervalo entre

¹<https://github.com/Bukkit/Bukkit>

²<https://github.com/soelynn/CoolReader>

³<https://github.com/google/google-auth-library-java>

⁴<https://github.com/caelum/guj.com.br>

⁵<https://github.com/caelum/mamute>

⁶<https://github.com/caelum/vraptor4>

commits, onde a mineração é realizada em todos os *commits* existentes entre o intervalo informado. Para a execução deste estudo, a segunda opção foi utilizada.

As evoluções analisadas neste estudo obedeceram o intervalo de commits entre a versão atual, no momento da análise do estudo, e a versão anterior de cada repositório selecionado.

4.1.2.2 Fase 2 - Execução da Mineração de Mudanças e das Análises Estática e Dinâmica

A execução da mineração de mudanças e das análises estática e dinâmica requerem que o código fonte do sistema a ser analisado esteja presente na máquina onde o estudo será realizado. A ferramenta possui um arquivo de configuração de onde são extraídas as informações sobre o sistema a ser analisado e as respectivas evoluções. O arquivo possui sete variáveis que devem ser modificadas de acordo com as informações específicas de cada sistema. A seguir tais variáveis são descritas.

REPOSITORY LOCAL PATH	O caminho do diretório local onde se encontra o código do repositório
GITHUB URL	O endereço online onde se encontra hospedado o repositório
EVOLUTION REPO LOCAL PATH	O caminho do diretório local onde se encontra o código do repositório com as evoluções
EVOLUTION BRANCH	O nome do branch onde as modificações foram realizadas
EVOLUTION PULL REQUEST	O número do Pull Request que contém as modificações a serem analisadas
EVOLUTION START VERSION	O número do commit inicial das mudanças a serem analisadas
EVOLUTION END VERSION	O número do commit final das mudanças a serem analisadas

Caso o cenário seja referente à análise de *Pull Requests*, as variáveis *EVOLUTION START VERSION* e *EVOLUTION END VERSION* não precisam ser atribuídas e se o cenário for de análise de um intervalo de *commits*, a variável *EVOLUTION PULL REQUEST* não precisa ser atribuída.

Para a análise dinâmica, é necessário que os testes do sistema sejam executados juntamente com a instrumentação do código da ferramenta que é responsável por capturar

os fluxos de execução que estão sendo executados pelos testes para um arquivo de texto.

4.1.2.3 Fase 3 - Análise dos Resultados

Como resultado, a ferramenta apresenta em um arquivo de texto um conjunto de fluxos de execução que foram afetados pelas mudanças e um conjunto de fluxos de execução que foram executados pelos testes, assim, o conjunto resultante da diferença entre os dois é o conjunto de fluxos de execução que foram impactados pelas mudanças que não estão sendo executados pelos testes.

Para análise dos resultados, foi feita uma contagem e tabulação da quantidade de fluxos de cada conjunto para cada cenário de evolução analisado. Como informação complementar, também foi feita uma contagem de quais módulos ou pacotes que mais foram afetados pelas modificações e que não foram cobertos pela suíte de testes.

4.1.3 Sistemas Analisados

Como dito anteriormente, ao todo, seis sistemas de código aberto foram selecionados para a execução deste estudo. A escolha dos sistemas se deu, além dos critérios já citados, pela diversidade tanto na área da aplicação quanto no tamanho dos mesmos. A seguir uma breve descrição de cada sistema, com informações coletadas em Novembro de 2015, é apresentada.

Bukkit Bukkit é um servidor tipo API (*Application Programming Interface*) que dá suporte à modificações no jogo Minecraft ⁷. O sistema contém 731 arquivos .java, 43075 linhas de código, 1509 commits e 131 contribuidores.

Cool Reader Cool Reader é uma biblioteca que transforma arquivos no formato csv (*Comma Separated Value*) para objetos Java. O projeto contém 21 arquivos .java, 908 linhas de código, 47 commits e 2 contribuidores.

Google Auth Library O Google Auth Library é um pequeno projeto da Google que dá suporte ao protocolo open source de autenticação OAuth para a linguagem Java. O sistema contém 30 arquivos, 163 linhas de código, 13 commits e 2 contribuidores.

GUJ O GUJ (Grupo de Usuários Java) é um sistema de fórum voltado para discussão de assuntos relacionadas à linguagem Java. O sistema contém 34790 linhas de código, 437 arquivos .java, 317 commits e 11 contribuidores.

⁷<https://minecraft.net/>

Mamute Mamute é uma *engine* que permite a criação de sistemas no estilo perguntas e repostas, semelhante ao *StackOverflow*⁸. O sistema contém 440 arquivos .java, 3539 commits, 17752 linhas de código e 22 contribuidores.

Vraptor Vraptor é um framework web do tipo *action-based*, desenvolvido a partir do CDI (*Contexts & Dependency Injection for Java*) e que atende ao padrão MVC. O sistema contém 25171 linhas de código, 3417 commits, e 64 contribuidores.

4.1.4 Métricas para Avaliação

Para avaliar a abordagem e responder as questões de pesquisa, a ferramenta analisou diversas evoluções dos sistemas selecionados. Analisando os resultados gerados pela ferramenta para cada evolução, é possível identificar exatamente quais fluxos de execução foram afetados e quais estão sendo testados ou não. A seguir são descritos os dados que foram gerados pela ferramenta de análise.

Métodos modificados (MM): é o número de métodos que foram modificados na evolução em análise.

Fluxos de chamadas existentes (FE): são os fluxos de chamadas existentes no sistema que foram obtidos pela análise estática.

Fluxos de chamadas afetados (FA): são os fluxos de chamadas capturados pela análise estática que foram afetados pelas mudanças ocorridas na evolução, ou seja, é o conjunto de fluxos de execução que contém os métodos modificados como elemento na cadeia de chamadas.

Fluxos de chamadas afetados cobertos (FC): são os fluxos de chamadas que foram afetados pelas mudanças ocorridas na evolução e que estão cobertos pela suíte de testes da aplicação que foram obtidos pela análise dinâmica.

Fluxos de chamadas não cobertos (FNC): é a diferença entre os dois conjuntos acima, ou seja, os fluxos de chamadas afetados pelas mudanças que não estão cobertos pela suíte de testes da aplicação.

Porcentagem de fluxos não cobertos (PC): é a relação entre a quantidade de FC com a quantidade de FA, ou seja, dentre os fluxos afetados, quantos foram cobertos. Quanto maior o número de PC melhor a qualidade da suíte de testes no que diz respeito à cobertura de fluxos modificados.

⁸<http://stackoverflow.com/>

4.2 Resultados Obtidos

Os resultados, obtidos a partir da execução do estudo descrito acima, são dispostos a seguir com o intuito de responder as questões de pesquisa do trabalho.

4.2.1 Visão Geral dos Resultados

Como resultado da execução do estudo acima, os resultados a seguir buscam responder à primeira questão de pesquisa (**RQ1**).

"Qual a porcentagem de fluxos de execução afetados pelas evoluções que não estão cobertos por suítes de testes automatizadas dentro dos projetos open-source analisados?"

A Tabela 1 apresenta os resultados obtidos de forma resumida, ou seja, os resultados de cada cenário de evolução analisado foram somados para se ter uma visão geral do que foi obtido. Uma análise mais detalhada dos resultados de cada cenário de evolução e mudanças realizadas são apresentadas a seguir na Seção 4.2.2.

Repositório	FE	MM	FA	FC	FNC	PC
Bukkit	3591	68	326	25	301	7,66%
Cool Reader	102	21	131	23	108	17,55%
Google Auth Library	300	66	211	150	61	71,09%
GUJ	3035	704	10001	3579	6422	35,78%
Mamute	2190	279	2634	1405	1229	53,34%
Vraptor	3324	56	900	723	177	80,33%

Tabela 1: Resultados da análise

FE: Fluxos de chamadas existentes

MM: Métodos modificados

FA: Fluxos de chamadas afetados

FC: Fluxos de chamadas afetados cobertos

FNC: Fluxos de chamadas não cobertos

PC: Porcentagem de cobertura

Como dito anteriormente, a Porcentagem de Cobertura é a relação entre o número de Fluxos Cobertos e a quantidade de Fluxos Afetados. Vale ressaltar que a *PC* calculada não leva em consideração a quantidade de Fluxos Existentes, portanto não pode ser comparada com a porcentagem de cobertura do sistema como um todo.

Com os resultados da Tabela 1, podemos afirmar que a maioria dos sistemas analisados não possuem uma boa cobertura de fluxos de execução do sistema, ou seja, as interações

entre os componentes do sistema não estão sendo cobertos devidamente, deixando uma brecha para introdução de novos bugs a partir de modificações no código que podem afetar diversas partes do sistema.

4.2.2 Resultados e Análise Detalhada de cada Sistema

Buscando medir a qualidade das suítes de testes dos sistemas analisados, uma análise sobre a evolução de *FNC* e *FC* foi realizada com o intuito de responder a segunda questão de pesquisa (**RQ2**).

"Considerando o histórico de evolução dos projetos, existe degradação ou melhoria da suíte de teste no que se refere a cobertura de fluxos modificados e não cobertos?"

A seguir, para cada sistema analisado, um gráfico de evolução de cobertura no que diz respeito aos fluxos afetados cobertos e não cobertos foi criado para ilustrar e determinar se houve uma melhoria ou degradação na suíte de testes no decorrer do tempo em que os sistemas foram analisados. A degradação ocorre quando o número de fluxos cobertos não co-evolve juntamente com o número de fluxos não cobertos, caracterizando a situação em que as evoluções no código acarretaram uma maior quantidade de fluxos não cobertos que não estão sendo cobertos pela suíte de testes.

Bukkit

Para o primeiro sistema analisado, Bukkit, para os 68 métodos que foram modificados nas 5 evoluções analisadas, a abordagem identificou que 326 fluxos de execução foram diretamente afetados pelas mudanças. Dos 326 fluxos afetados, a ferramenta, através da análise dinâmica na execução da suíte de testes, identificou que 25 fluxos estavam sendo testados. Assim, a abordagem conseguiu identificar que 301 fluxos de execução não estão sendo testados pelo sistema. Uma listagem com uma amostra dos fluxos de execução identificados é disposta no Apêndice A.

Ao todo, 5 cenários de evolução foram analisados, a Tabela 2 apresenta os resultados obtidos para cada cenário analisado. Pode-se perceber que dentre os cenários de evolução analisados, em apenas dois cenários a análise dinâmica conseguiu identificar fluxos de execução que já se encontravam cobertos pelos testes.

Cenário	MM	FA	FC	FNC	PC
1	11	40	14	25	35%
2	19	42	5	37	11,90%
3	13	63	0	63	-
4	15	68	0	68	-
5	10	113	6	107	5,3%

Tabela 2: Resultados da análise pra cada Cenário de Evolução do Sistema Bukkit

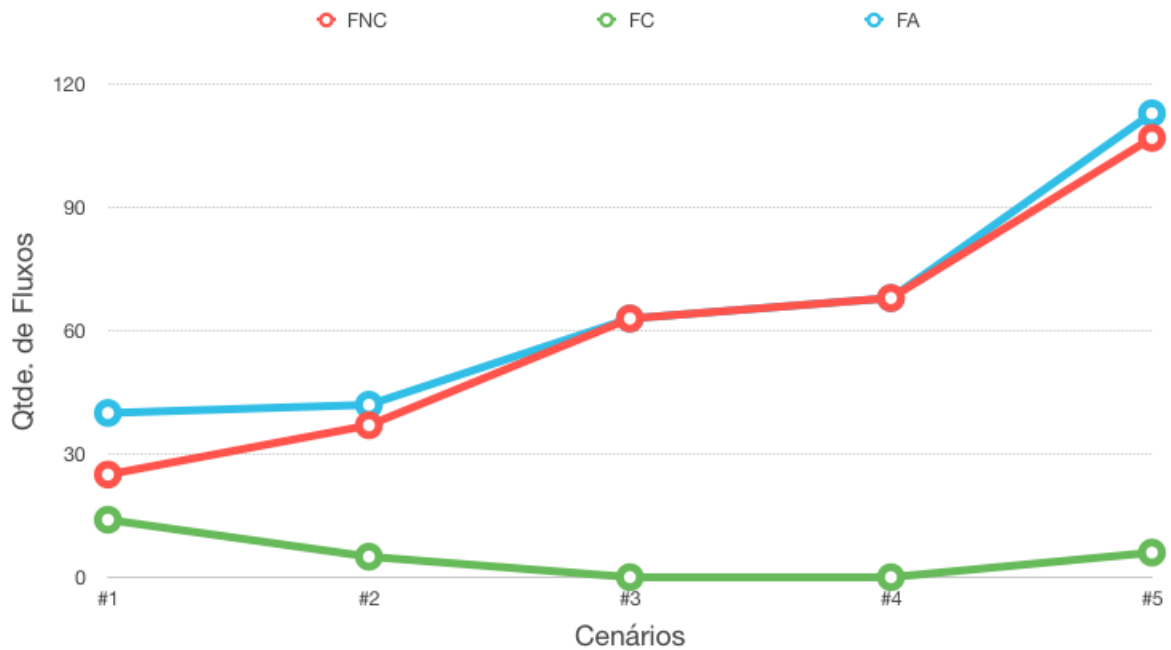


Figura 11: Evolução de Cobertura do Sistema Bukkit

Na Figura 12, que ilustra a evolução na cobertura do sistema Bukkit, é possível determinar que houve uma degradação na qualidade da suíte de testes no que diz respeito aos fluxos de execução afetados, o que era o esperado, uma vez que os fluxos cobertos não evoluíram conjuntamente com as modificações no código.

A Figura 13 apresenta um diagrama de pacotes com o intuito de ilustrar o impacto das evoluções no que diz respeito a cobertura de fluxos de execução. Pode-se perceber que dos 8 pacotes afetados pelas evoluções, apenas 3 possuem uma incidência maior de fluxos de execução cobertos pelos testes. A maior parte dos fluxos afetados e analisados originam-se dos pacotes *Command* e *Configuration*.

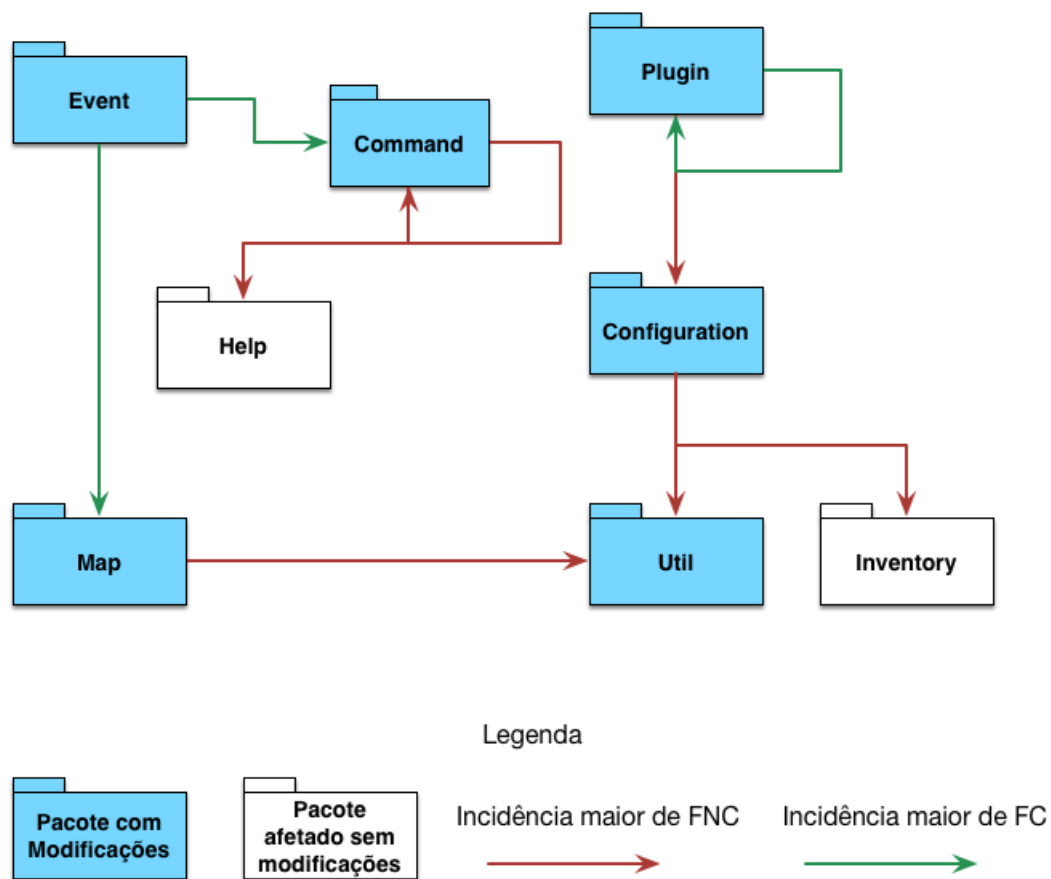


Figura 12: Diagrama de Impacto do Sistema Bukkit

Cool Reader

A Tabela 3 apresenta os resultados obtidos para o segundo sistema analisado, o Cool Reader. Ao todo, 21 métodos foram analisados nas 2 evoluções selecionadas, resultando na identificação de 108 fluxos de execução que foram afetados pelas mudanças e não estão sendo acessados pelos métodos de teste.

Cenário	MM	FA	FC	FNC	PC
1	5	46	8	38	17,39%
2	16	85	17	68	20%

Tabela 3: Resultados da análise pra cada Cenário de Evolução do Sistema Cool Reader

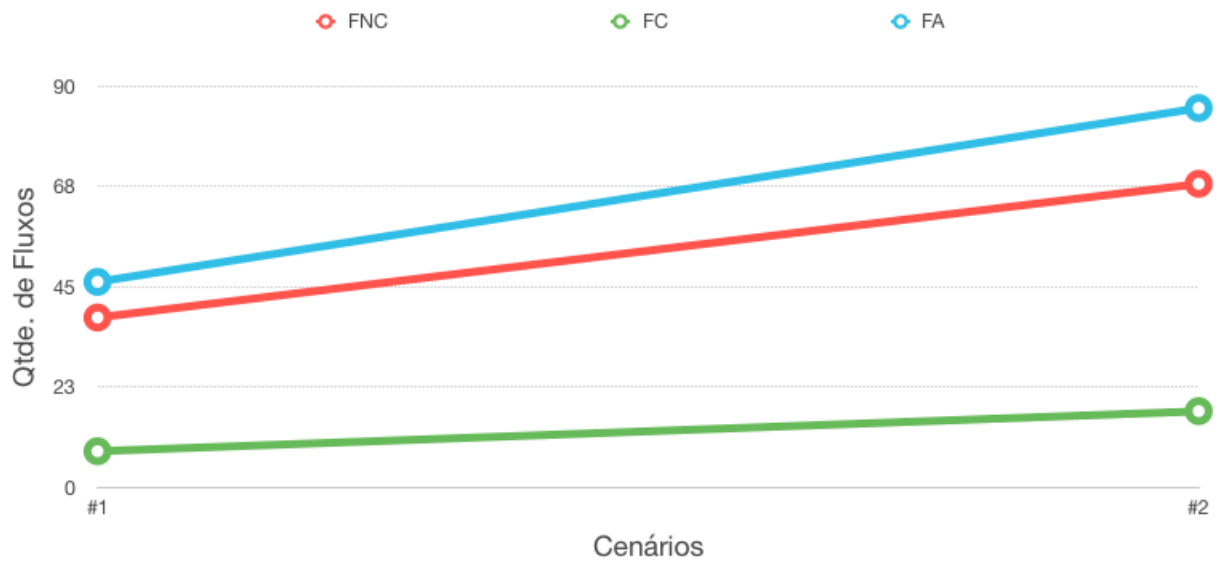


Figura 13: Evolução de Cobertura do Sistema Cool Reader

Na Figura 14 é apresentada a evolução de cobertura do sistema a partir dos resultados obtidos pela execução da abordagem. Como o número de cenários analisados para o *Cool Reader* é baixo, não é possível determinar se houve ou não degradação da suíte de testes com os resultados obtidos. O que podemos concluir é que houve uma co-evolução entre os dados obtidos.

O diagrama de pacotes para o sistema Cool Reader é apresentado na Figura 15. Pelo diagrama é possível perceber que apenas 2 pacotes (*Datasource* e *Annotation*) possuem um índice de FC maior que o número de FNC encontrado. Grande parte dos fluxos analisados originam-se do pacote *coolreader*.

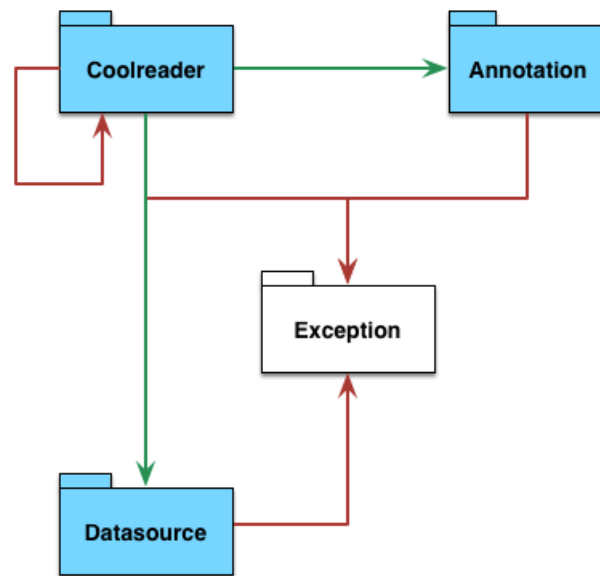


Figura 14: Diagrama de Impacto do Sistema Cool Reader

Google Auth Library

A Tabela 4 apresenta os resultados obtidos para o terceiro sistema analisado, o Google Auth Library, que por se tratar de uma aplicação bem pequena e mais simples que a anterior, a abordagem conseguiu identificar 61 fluxos de execução que não estão sendo testados, dentre os 211 fluxos afetados pelas mudanças nos 48 métodos modificados.

Cenário	MM	FA	FC	FNC	PC
1	9	47	46	1	97,87%
2	17	16	9	7	56,25%
3	5	23	10	13	43,47%
4	3	22	9	13	40,90%
5	14	103	76	27	73,78%

Tabela 4: Resultados da análise pra cada Cenário de Evolução do Sistema Google Auth Library

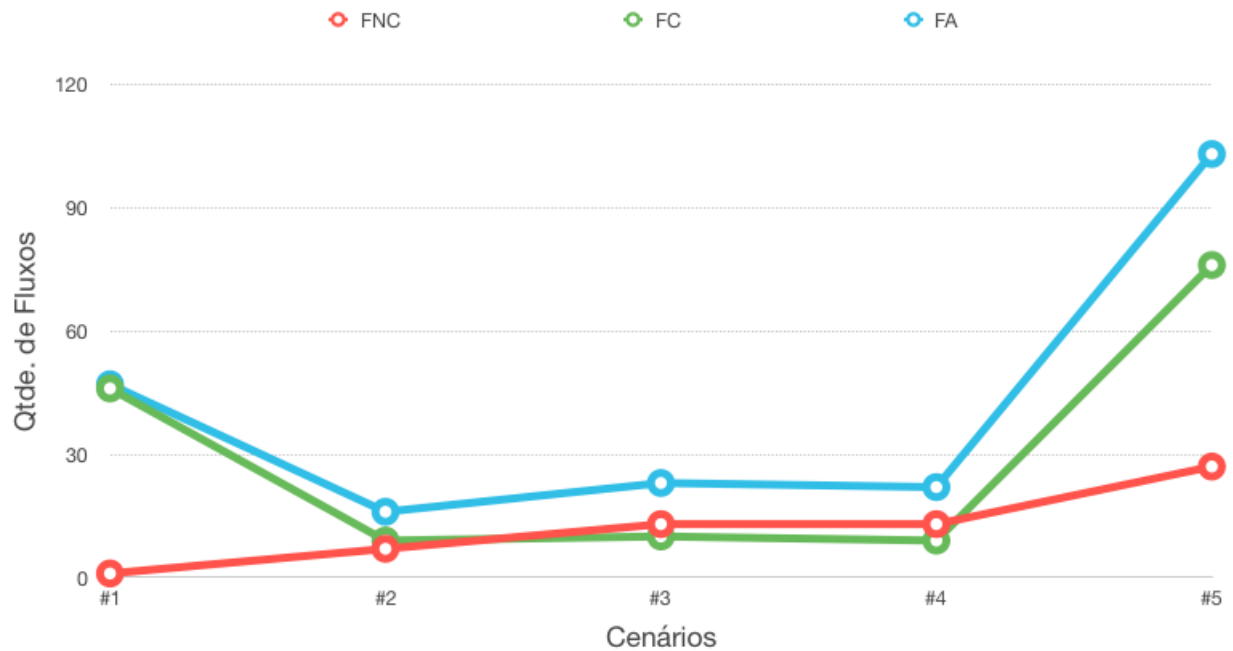


Figura 15: Evolução de Cobertura do Sistema Google Auth Library

Na Figura 16 é possível perceber que o número de *FC* co-evolve juntamente com as evoluções, o que caracteriza uma melhoria na suíte de testes no que diz respeito aos fluxos de execução afetados e cobertos.

O Sistema Google Auth Library possui apenas 2 pacotes de código, *Oauth2* e *Http*. A maioria dos fluxos de execução identificados a partir das mudanças foram do pacote *Oauth2*, que acessa diretamente o pacote *Http* para executar as funcionalidades de autenticação. O diagrama de pacotes, apresentado na Figura 17, é mais apropriado para sistemas mais complexos, mas a ilustração demonstra que o número de *FC* é maior que o número de *FNC* em ambos os pacotes existentes, comprovando um bom índice de cobertura que o sistema possui.

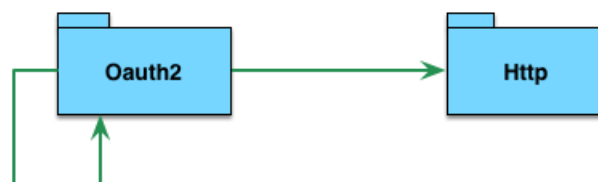


Figura 16: Diagrama de Impacto do Sistema Google Auth Library

A Tabela 5 apresenta os resultados obtidos para o quarto sistema analisado, o GUJ. Nos 7 cenários analisados, 704 métodos modificados foram identificados, resultando na análise de 10001 fluxos afetados pelas mudanças. Dentre os 10001 fluxos afetados, 3579 fluxos foram executados pelos testes, deixando 6422 fluxos de execução não cobertos pela execução dos testes.

Cenário	MM	FA	FC	FNC	PC
1	98	991	509	482	51,36%
2	59	1207	384	823	31,81%
3	78	632	251	381	39,71%
4	119	1256	465	791	37,02%
5	88	2047	824	1223	40,25%
6	167	2154	641	1513	29,75%
7	95	1714	505	1209	29,46%

Tabela 5: Resultados da análise pra cada Cenário de Evolução do Sistema GUJ

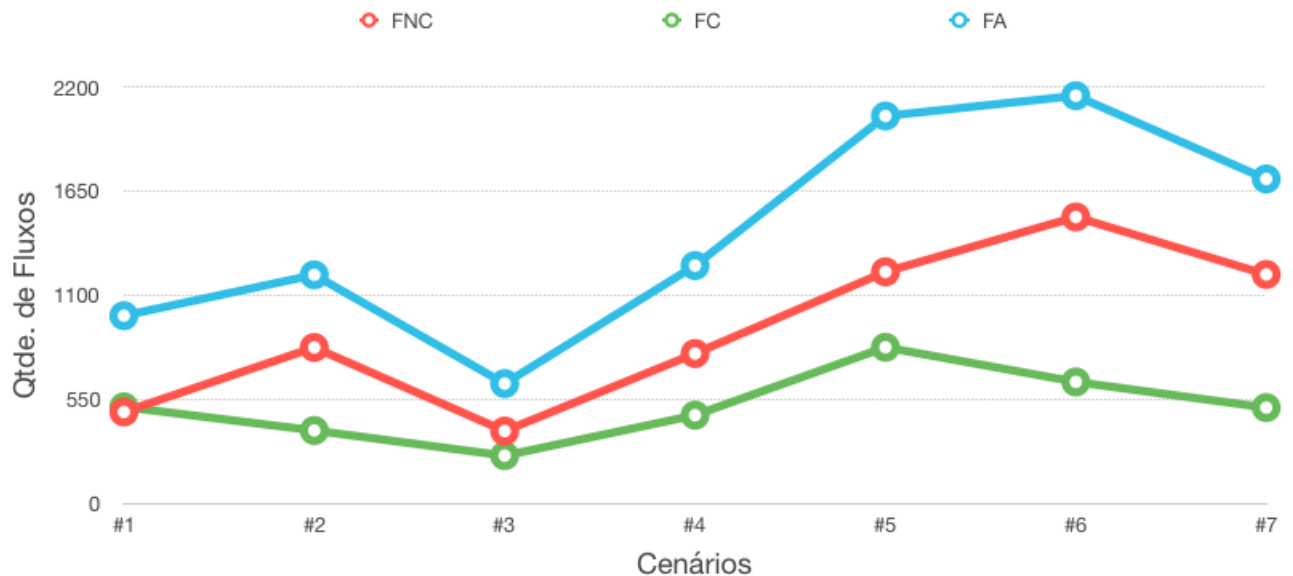


Figura 17: Evolução de Cobertura do Sistema GUJ

Na Figura Figura 18, podemos perceber que em todos os cenários analisados, a quantidade de FNC sempre é maior que a quantidade de FC, o que indica que as mudanças nos testes não foram suficientes para suprir os impactos colaterais que foram introduzidos pelas evoluções.

No diagrama de pacotes para o sistema GUJ, Figura 19, pode-se perceber que as mudanças contidas nas evoluções estão espalhadas por vários pacotes da aplicação, mas apenas os fluxos de execução oriundos dos pacotes *Controller*, *Dao* e *Model* obtiveram

um bom índice de cobertura. Um grande número dos fluxos de execução tidos como FNC são oriundos dos pacotes *Feeds*, *Filter* e *Uri*.

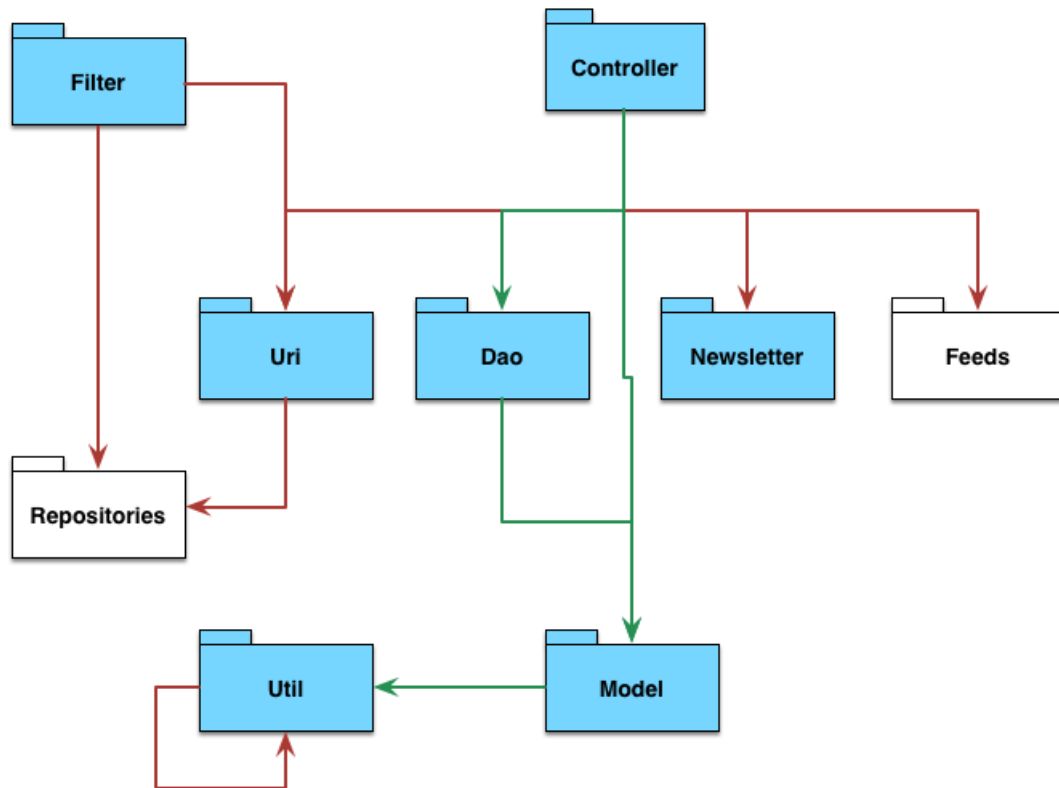


Figura 18: Diagrama de Impacto do Sistema GUI

Mamute

A Tabela 6 apresenta os resultados obtidos para o quinto sistema analisado, o Mamute. Ao todo, 279 métodos foram identificados como modificados dentro das 4 evoluções analisadas. No total, 2634 fluxos de execução foram afetados pelas mudanças, deste total, 1405 fluxos são executados pelos testes, deixando 1229 fluxos de execução não cobertos.

Cenário	MM	FA	FC	FNC	PC
1	61	658	362	296	55,01%
2	49	542	304	238	56,08%
3	75	821	493	328	60,04%
4	94	613	246	367	40,13%

Tabela 6: Resultados da análise pra cada Cenário de Evolução do Sistema Mamute

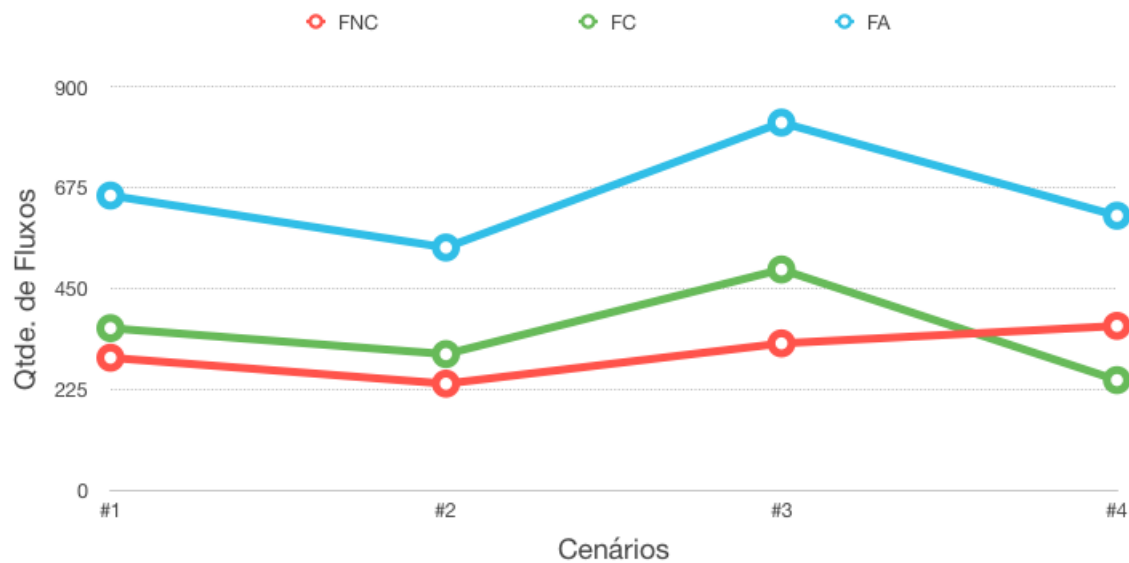


Figura 19: Evolução de Cobertura do Sistema Mamute

Na Figura 20, é possível perceber que apenas no último cenário houve uma degradação da suíte de testes no que diz respeito a cobertura de fluxos afetados, o que indica que o sistema conseguiu evoluir bem a suíte de testes no decorrer das evoluções de código.

O diagrama de pacotes do Mamute, apresentado na Figura 21, ilustra quais pacotes foram afetados pelas mudanças. Pode-se perceber que há uma atenção maior para com a cobertura de testes nos pacotes que acessam o pacote *Model*, deixando vários outros pacotes sem a cobertura devida. A maioria dos fluxos analisados para o sistema Mamute originam-se dos pacotes *Controller* e *Auth*.

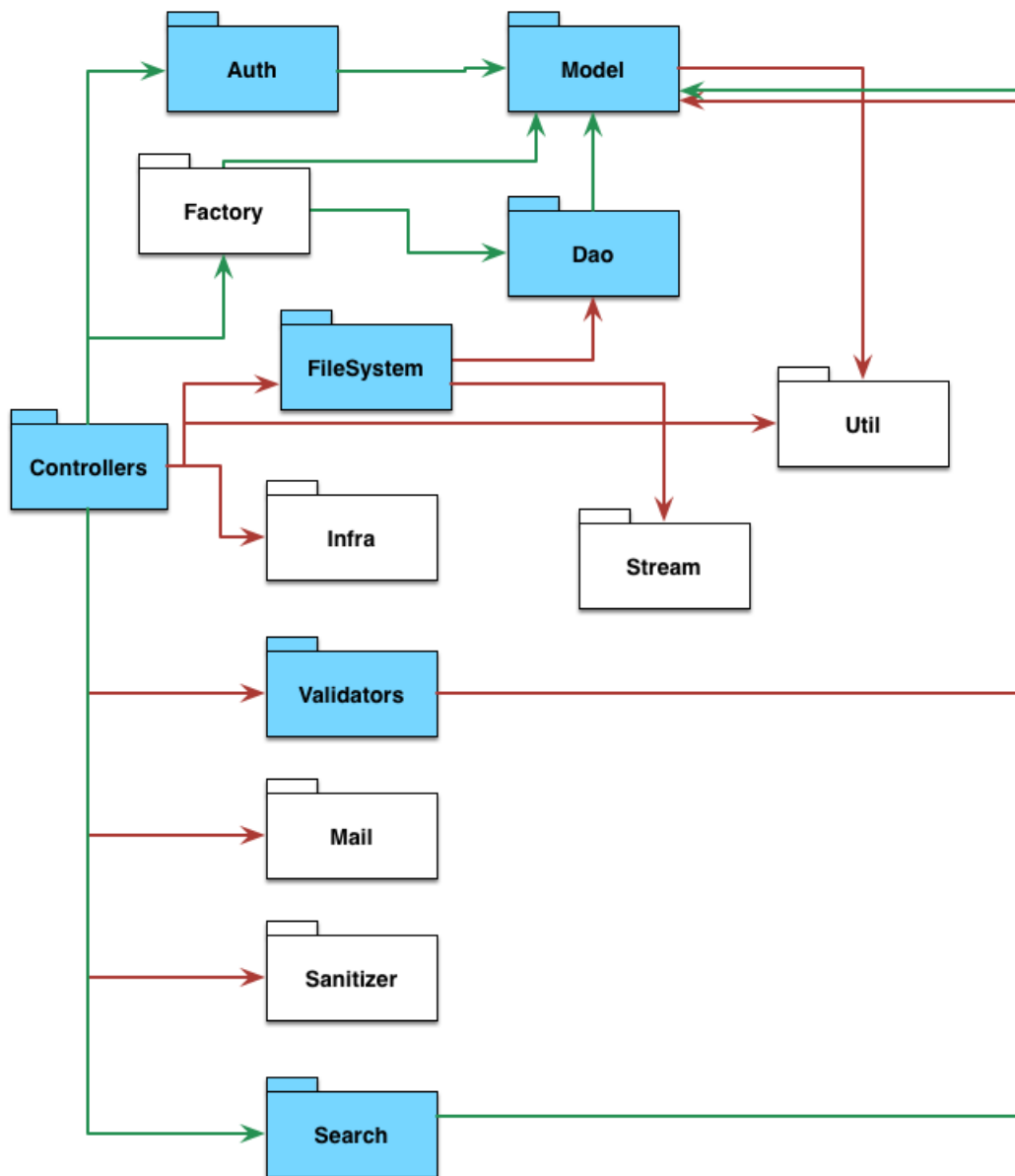


Figura 20: Diagrama de Impacto do Sistema Mamute

VRaptor

A Tabela 7 apresenta os resultados obtidos para o sexto e último sistema analisado, o VRaptor. Das 5 evoluções analisadas, 56 métodos foram identificados como modificados, a modificação destes 56 métodos resultaram no impacto de 900 fluxos de execução. Pela análise dinâmica, foi encontrado que 723 destes 900 fluxos já se encontram cobertos pelos testes, deixando 177 fluxos de execução não cobertos pelos testes.

Cenário	MM	FA	FC	FNC	PC
1	13	302	193	110	63,90%
2	20	148	117	31	79,05%
3	7	16	11	5	68,75%
4	8	55	55	0	100%
5	8	379	347	31	91,55%

Tabela 7: Resultados da análise pra cada Cenário de Evolução do Sistema VRaptor

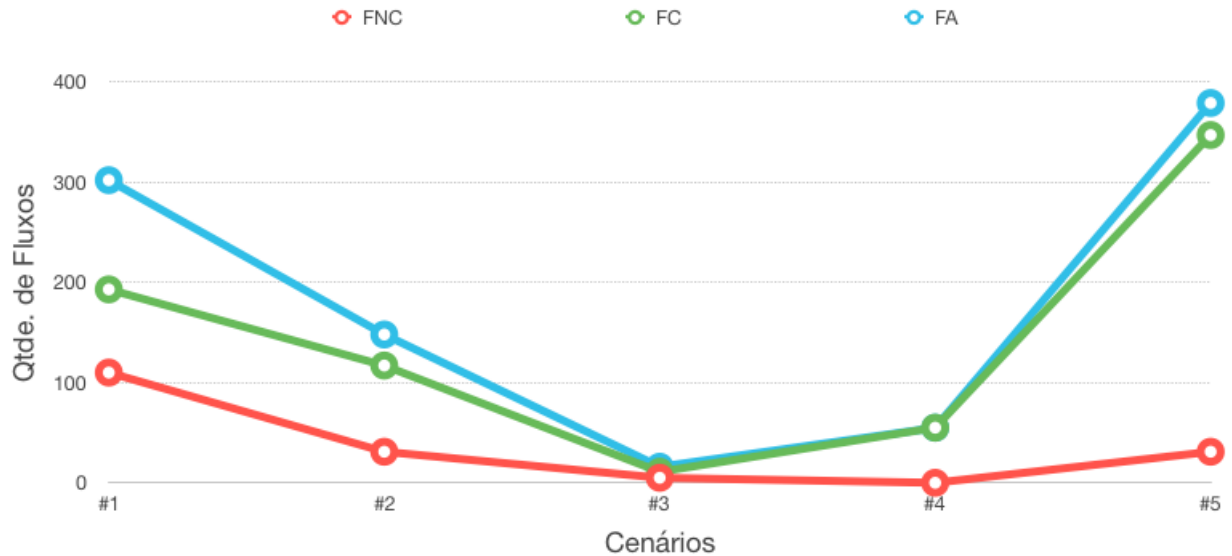


Figura 21: Evolução de Cobertura do Sistema VRaptor

O gráfico de cobertura do sistema VRaptor, apresentado na Figura 22, demonstra que não houve uma degradação na suíte de testes, e sim uma melhoria na mesma.

No diagrama de pacotes para o sistema VRaptor, Figura 23, é possível perceber que há uma boa cobertura de grande parte do sistema. Dentre os pacotes que houveram modificações, apenas três pacotes, *Core*, *Validator* e *Util*, obtiveram um maior número de FNC. A grande maioria dos fluxos de execução analisados pela abordagem para o sistema VRaptor originam-se dos pacotes *Controller* e *Core*.

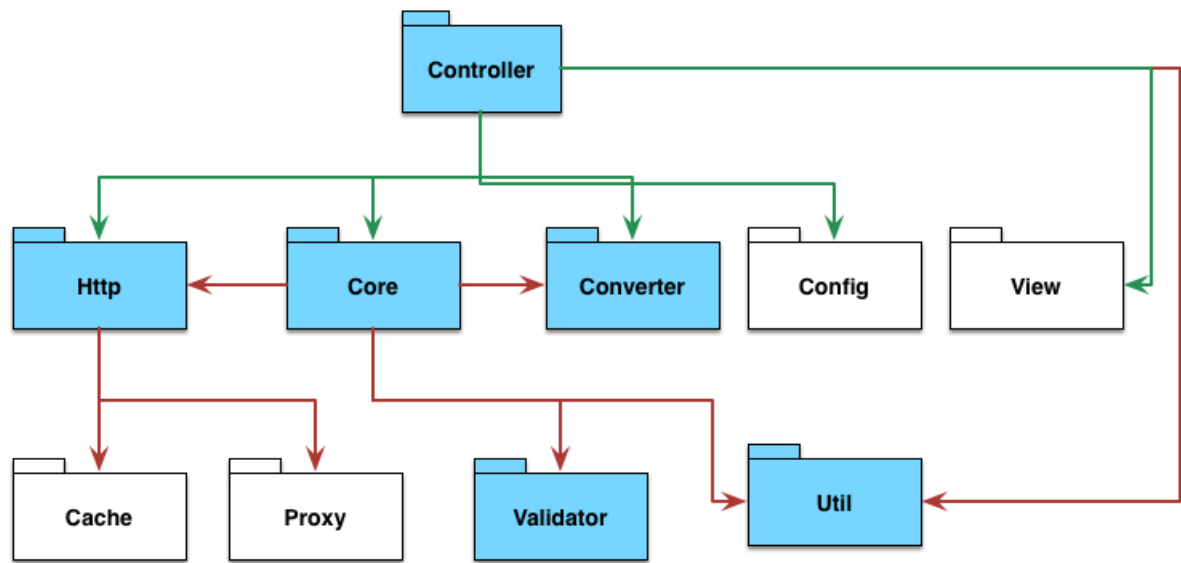


Figura 22: Diagrama de Impacto do Sistema VRaptor

5 Trabalhos Relacionados

Este capítulo apresenta trabalhos de pesquisa no contexto de: (i) cobertura e análise de código (Seção 5.1), (ii) análise do impacto de mudanças (Seção 5.2), e (iii) testes de regressão (Seção 5.3). No decorrer do capítulo, as características de cada trabalho são destacadas, bem como são realizadas comparações entre os trabalhos relacionados e a abordagem proposta na dissertação.

5.1 Abordagens de Cobertura e Análise de Código

Em (OOSTERWAAL, 2015), o autor desenvolveu uma ferramenta de análise de cobertura de testes, intitulada Operias, que analisa o índice de cobertura de cada arquivo de código antes e depois de evoluções que estão contidas em *pull requests* de repositórios Java que estejam hospedados no Github. O trabalho analisa, baseado em relatórios de cobertura da ferramenta Cobertura¹, se as modificações contidas nos pull requests melhoram ou pioram o índice de cobertura do código e mostra ainda quais linhas de código foram afetadas pelas modificações.

O trabalho se destaca pela integração da análise de cobertura de código, obtida por uma solução já consolidada e validada (YANG; LI; WEISS, 2009; ALEMERIEN; MAGEL, 2014; SHAHID; IBRAHIM, 2011a), com as evoluções contidas nos pull requests do github. Por estar se tornando a solução mais escolhida para hospedagem de sistemas open source, o Github é objeto de pesquisa para diversos trabalhos, que mineram sua base de dados com os mais diversos objetivos (KALLIAMVAKOU et al., 2014).

Como resultado, a ferramenta se mostra bastante promissora por suprir a carência de ferramentas que analisam a cobertura de código de repositórios hospedados e evoluídos no github. Apesar de a ferramenta estar inserida no contexto de análise de cobertura de código e extrair as evoluções a serem analisadas do github, a abordagem adotada pelo

¹<http://cobertura.github.io/cobertura/>

autor utiliza as métricas de estado e branch para determinar os índices de cobertura, não sendo possível assim a confrontação dos resultados.

No trabalho de (MARINESCU; HOSEK; CADAR, 2014), um framework semelhante a ferramenta de Oosterwaal foi desenvolvido, intitulado de Covrig, ele analisa cada versão isolada de um sistema, usando técnicas de análise de código tanto estáticas como dinâmicas em busca por informações que retratem como os códigos e os seus respectivos testes evoluem conjuntamente. A ferramenta foi aplicada em diversas versões de seis grandes sistemas desenvolvidos na linguagem C (GNU Binutils, Git, Lighttpd, Memcached, Redis e ØMQ) em busca de informações que respondessem perguntas sobre como as evoluções afetam o estado do sistema. O trabalho adotou a utilização de contêineres de software de baixo custo para distribuir e paralelizar a análise do código (SOLTESZ et al., 2007). Maiores detalhes sobre as técnicas de análise de código utilizadas não foram apresentadas, é dito apenas que a análise estática foi feita diretamente no código fonte e a análise dinâmica foi realizada a partir da execução dos testes de regressão dos sistemas.

Como a métrica utilizada para determinar os índices de cobertura utilizada pelo trabalho foi no âmbito de linhas de código (*line mapping*), os resultados não puderam ser comparados diretamente, uma vez que os resultados obtidos divergem, ou seja, quando a abordagem utiliza a métrica de linhas de código, por exemplo, o resultado será quais linhas de código estão ou não cobertas pelos testes, o que difere dos resultados obtidos nessa abordagem, que procura pelos fluxos de chamadas não cobertos pelos testes.

5.2 Abordagens de Análise do Impacto de Mudanças

Alguns autores, em trabalhos de sumarização (ARNOLD; BOHNER, 1993; BOHNER, 2002; LI et al., 2013), dividem os estudos da área de análise do impacto de mudanças em três grandes grupos, os trabalhos que utilizam a análise estática, os trabalhos que utilizam a análise dinâmica e os trabalhos que combinam as duas análises para prover os resultados. A abordagem aqui apresentada, se insere no terceiro grupo, pois ela utiliza (i) a análise estática para obter os fluxos de execução de todo o sistema em análise e (ii) obtém os fluxos de execução que estão sendo executados pelos testes a partir de uma análise dinâmica.

Grande parte dos trabalhos da área possuem como objetivo geral *identificar um conjunto de entidades do sistema que potencialmente foram afetadas por mudanças no código*. Algumas abordagens vão além e buscam apresentar juntamente com as entidades poten-

cialmente afetadas, *quais mudanças que geraram esse impacto*. Primeiramente, algumas abordagens que utilizam a análise estática serão discutidas, depois uma discussão sobre as abordagens que utilizam a análise dinâmica também será apresentada e por fim as abordagens que usam a combinação das duas análises serão discutidas.

Uma das primeiras abordagens de análise de impacto de mudanças, apresentada por (BOHNER; ARNOLD, 1996), utilizava a acessibilidade em um grafo de chamadas para medir o impacto das mudanças. Apesar de ter sido o ponto de partida para muitas outras ferramentas de análise de impacto, a técnica é considerada imprecisa por considerar apenas os métodos filhos, no grafo de chamadas, como afetados pelas mudanças, ou seja, os métodos *callers* dos métodos modificados, que também podem ser afetados, não são considerados.

O trabalho de Law e Rothermel (LAW; ROTHERMEL, 2003) apresenta uma abordagem, intitulada PATHIMPACT, que a partir de código instrumentado (análise dinâmica) e a adoção da técnica de *forward slicing* (definida na Seção 1.2) identifica os fluxos de execução afetados pelas evoluções do sistema. A abordagem tem como principal objetivo prever, com uma maior precisão e eficiência, quais fluxos de execução serão afetados pelas modificações. O problema levantado pelos autores é o de que as análises estáticas são mais imprecisas e ineficientes, do que as análises dinâmicas, no que diz respeito a determinar precisamente (sem falso positivo) os conjuntos de fluxos de execução afetados por evoluções.

Para o contexto de análise de impacto de mudanças, os resultados do trabalho de (LAW; ROTHERMEL, 2003) foram satisfatórios, pois a abordagem conseguiu identificar um maior número de fluxos do que outras abordagens e um número menor de fluxos se comparado com o resultado proveniente de abordagens que utilizaram a técnica *static slicing*, o que é o esperado, uma vez que a técnica de análise estática é mais abrangente e menos precisa que a técnica de análise dinâmica. Não foi possível fazer uma comparação dos resultados do trabalho com os resultados dessa dissertação uma vez que os objetivos diferem, mas o trabalho citado apresenta uma metodologia de experimentação que foi posta em prática na abordagem aqui proposta.

A abordagem de (ORSO; APIWATTANAPONG; HARROLD, 2003) realiza a análise de mudanças a nível de método e determina, a partir de uma combinação das análises de código estática e dinâmica, quais fluxos de execução foram impactados pelas mudanças. O trabalho também identifica se os fluxos de execução estão ou não sendo testados pela suíte de testes, assim como a abordagem aqui proposta. No entanto, o estudo difere deste em alguns pontos importantes, (i) a identificação do conjunto de fluxos de execução afetados pelas mudanças é realizada a partir de uma análise dinâmica, ou seja, é necessário

que haja uma instrumentação específica no código e que o sistema seja executado por algum usuário, o que restringe a análise à apenas o código que foi executado, diferente da nossa abordagem que utiliza a análise estática para esta etapa e não requer nenhum tipo de instrumentação e execução por parte do usuário, o que abrange o nicho de análise para bibliotecas, por exemplo; (ii) o conjunto de fluxos de execução afetados que são identificados pela abordagem de (ORSO; APIWATTANAPONG; HARROLD, 2003) é obtido a partir da técnica de *forward slicing*, que difere da nossa abordagem que utiliza a técnica de *backward slicing*; (iii) por último, os trabalhos diferem no que se trata à etapa de desenvolvimento para que elas foram desenvolvidas e portanto são mais adequadas, o trabalho aqui proposto é voltado para a etapa de desenvolvimento, uma vez que requer apenas o código fonte para realizar a análise, já o trabalho em comparação é voltado para a etapa de produção (*in field*), ou seja, tem como objetivo analisar apenas o que é executado pelo usuário.

Badri (BADRI; BADRI; ST-YVES, 2005) propõe uma abordagem de análise de impacto de mudanças que aplica a análise estática de código para construir o que os autores chamaram de grafo de controle de chamadas (*Control Call Graph*) que é definido como:

"Um grafo de controle de chamadas é um grafo de controle de fluxo em que os vértices que não representam chamadas são desconsiderados."

O trabalho traz como diferencial a extensão do modelo tradicional de análise em grafos de chamadas com a inclusão do controle do fluxo, modelo esse que se apresentou mais preciso que o modelo tradicional. Os resultados, apesar de serem preliminares, se mostraram promissores, em média, os fluxos tidos como impactados pelas mudanças usando o modelo proposto se mostrou 42% menor que o resultado usando o modelo tradicional. Apesar de apresentar um número bem menor de fluxos, na investigação e inspeção do código fonte, o modelo proposto se apresentou mais preciso por não apresentar falsos positivos.

5.3 Abordagens de Testes de Regressão

Ryder e Tip usam a análise de impacto para identificar quais mudanças foram responsáveis por alterar o comportamento dos testes de regressão de um sistema (RYDER; TIP, 2001). A técnica proposta consiste em transformar o código fonte das mudanças em um conjunto de mudanças atômicas, permitindo que um grafo de relacionamento entre as mudanças seja construído para uma posterior análise e identificação das mudanças que afetaram o comportamento dos testes.

Posteriormente, Ryder et al constroem uma ferramenta, denominada de *Chianti*, para validar a técnica de análise de impacto com foco em testes de regressão (REN et al., 2004). Neste trabalho, além da análise de impacto estática, também é empregada a análise de impacto dinâmica. O Chianti recebe como entrada o programa original, o programa modificado e o conjunto de testes de unidade, e fornece os testes afetados e o conjunto de mudanças que afetaram os testes como saída. Além do conjunto de mudanças atômicas, o Chianti usa grafos de chamadas, que podem ser obtidos através de análise estática, ou através da execução dos testes de regressão (análise dinâmica), para realizar a análise de impacto. O Chianti foi submetido a um estudo de caso, no qual se realizou uma análise do sistema Daikon, escrito em Java. Destaca-se o comparativo dos resultados da análise de impacto usando grafos de chamadas obtidos através da análise estática e da execução de testes de regressão (análise dinâmica), dos quais o último obteve resultados mais precisos.

O Chianti e o trabalho aqui proposto se assemelham-se por utilizarem o grafo de chamadas como técnica de análise de impacto. No entanto, o objetivos diferem uma vez que o Chianti apresenta os testes potencialmente afetados e a abordagem desta dissertação apresenta os potenciais fluxos de execução afetados e não cobertos pelos testes.

Não realizamos uma análise comparativa de nossa técnica com as demais abordagens da área, devido à falta de disponibilização de ferramentas ou de detalhes dos algoritmos suficientes para que uma implementação fosse possível. Os trabalhos que apresentaram maiores detalhes acerca dos algoritmos propostos, são complexos e demandariam um grande esforço. Portanto, sugerimos que futuramente seja realizada uma análise comparativa em termos de resultados obtidos e desempenho.

6 Conclusão

Este trabalho apresentou uma abordagem que tem como principal objetivo a melhoria da suíte de testes automatizados através da identificação de possíveis fluxos de execução que foram afetados por evoluções e que não estão sendo alcançados pelos testes disponíveis. Uma ferramenta para automatizar a execução da abordagem proposta foi implementada utilizando as linguagens Java e AspectJ. A ferramenta permite uma análise das evoluções de código de sistemas Java utilizando uma combinação das análises de código estática e dinâmica. Diferente das abordagens existentes, a abordagem utiliza a técnica *backward slicing* para identificar os fluxos de execução afetados pelas modificações no código. Com os resultados obtidos, espera-se que as equipes de desenvolvimento e de testes obtenham uma melhor cobertura de código, considerando fluxos de execução que antes não estavam sendo propriamente testados.

O restante do capítulo está organizado da seguinte forma: A Seção 6.1 apresenta as principais contribuições do trabalho; A Seção 6.2 discute as limitações da abordagem, ferramenta e do estudo realizado; Na Seção 6.3, algumas perspectivas de trabalhos futuros são apresentadas.

6.1 Principais Contribuições

As principais contribuições deste estudo são: (i) a abordagem proposta e sua implementação, que analisa o impacto que evoluções de código possuem sobre a cobertura de testes automatizados no nível de fluxos de execução, provendo uma informação que pode ser usada para melhorar a qualidade da suíte de testes e (ii) os resultados obtidos pela ferramenta permitem determinar se a suíte de testes está acompanhando ou não as evoluções de código.

Em resumo, o estudo encontrou que: (i) 92,33%, 82,45%, 28,91%, 64,22%, 46,66% e 19,67% dos fluxos de execução afetados não estão sendo acessados pelos testes automatizados, para os sistemas Bukkit, Cool Reader, Google Auth Library, GUJ, Mamute e

VRaptor, respectivamente, o que indica que dentre os projetos analisados, a maior parte deles possuem uma baixa cobertura de testes para fluxos de execução afetados por mudanças; (ii) os módulos de: *Command* e *Configuration*; *coolreader*; *Oauth2*; *Feeds*, *Filter* e *Uri*; *Controller* e *Auth*; e *Controller* e *Core*, foram os que mais tiveram ocorrências de fluxos de execução não cobertos, para os sistemas Bukkit, Cool Reader, Google Auth Library, GUJ, Mamute e VRaptor, respectivamente; (iii) em três dos seis sistemas (Bukkit, GUJ e Mamute), foi possível identificar, a partir do resultados do estudo, que houve uma degradação da suíte de testes no que diz respeito a cobertura de fluxos de execução afetados por mudanças de código.

6.2 Limitações da Abordagem

Esta seção apresenta limitações tanto da abordagem em si quanto das técnicas utilizadas e tecnologias envolvidas no desenvolvimento da ferramenta. Uma das principais limitações da abordagem proposta diz respeito à recursão de código. Foi observado um demasiado consumo de tempo e espaço na análise de sistemas que utilizam esta técnica de programação em um grau elevado, o que dificulta a obtenção e análise de resultados por parte da ferramenta. Uma outra limitação no processo de criação dos fluxos de execução é o uso da reflexão, que foi desconsiderada na análise dos sistemas.

Como já discutido (Seção 3.1), a análise dinâmica da abordagem é baseada na instrumentação automática da execução de testes que foram desenvolvidos com o framework JUnit, o que limita o escopo da análise para apenas os sistemas que tenham sido desenvolvidos com tal suporte.

Para a análise estática, a técnica de *backward slicing* adotada, também oferece uma limitação na identificação dos fluxos de execução, pois a abordagem considera apenas os fluxos que são executados após a execução dos métodos modificados, desconsiderando os os métodos que foram executados previamente.

Quanto à execução da ferramenta, existe a limitação de possuir a IDE Eclipse instalada, pois é a partir do Eclipse que a ferramenta é executada e configurada, assim como o banco de dados PostgreSQL, que é o responsável por armazenar os dados da análise dinâmica.

6.3 Trabalhos Futuros

Como trabalhos futuros, algumas direções para melhorias da abordagem proposta são apresentadas a seguir.

Visualização dos resultados

Os resultados obtidos pela abordagem e ferramenta desenvolvida são puramente arquivos de texto contendo os fluxos de execução identificados. Como trabalho futuro, uma forma de visualização gráfica dos fluxos de execução, apresentando as entidades (pacotes e classes) e seus relacionamentos, se apresenta possível.

Validação dos resultados e entrevista com os desenvolvedores

Uma validação dos resultados e entrevista para determinar o potencial da abordagem e dos resultados obtidos pode ser feita com a equipe de desenvolvimento dos sistemas analisados. Com a entrevista pode-se também buscar respostas para questões acerca de uma melhor forma para visualização dos resultados, um possível uso de ferramentas semelhantes, como a ferramenta poderia ajudar no processo de desenvolvimento, etc.

Condução de novos estudos empíricos

Uma vez que as evoluções e características de cada sistema diferem em vários aspectos, novos estudos empíricos, com uma maior variedade de sistemas e um maior detalhamento da execução e resultados podem também ser alvos de trabalhos futuros.

Condução de um estudo comparativo

Como mencionado no Capítulo 5, a comparação da abordagem com outros não é uma tarefa simples uma vez que as técnicas utilizadas diferem em vários aspectos. Contudo, um estudo comparativo entre as abordagens que apresentam como resultado os fluxos de execução afetados pelas mudanças pode ser feito a partir dos resultados apresentados no Capítulo 4.2. O estudo deve então aplicar as abordagens em um ambiente controlado onde os resultados poderão ser confrontados.

Condução de um estudo de avaliação

Além da validação dos resultados com as equipes de desenvolvimento dos sistemas analisados, uma avaliação da eficácia da abordagem na melhoria das suítes de testes através da identificação dos fluxos não cobertos se faz necessária, ou seja, mensurar a melhoria na qualidade de cobertura da suíte de testes a partir dos resultados obtidos pela abordagem.

Referências

- ALEMERIAN, K. A. Evaluation of Software Testing Coverage Tools : An Empirical Study. n. May, 2013.
- ALEMERIEN, K.; MAGEL, K. Examining the effectiveness of testing coverage tools: An empirical study. *International Journal of Software Engineering and Its Applications*, v. 8, n. 5, p. 139–162, 2014.
- AMMANN, P.; OFFUTT, J. *Introduction to software testing*. [S.l.]: Cambridge University Press, 2008.
- APIWATTANAPONG, T.; ORSO, A.; HARROLD, M. J. Efficient and precise dynamic impact analysis using execute-after sequences. In: *Proceedings of the 27th International Conference on Software Engineering*. New York, NY, USA: ACM, 2005. (ICSE '05), p. 432–441. ISBN 1-58113-963-2. Disponível em: <<http://doi.acm.org/10.1145/1062455.1062534>>.
- ARNOLD, R.; BOHNER, S. Impact analysis-towards a framework for comparison. In: *Software Maintenance ,1993. CSM-93, Proceedings., Conference on*. [S.l.: s.n.], 1993. p. 292–301.
- BADRI, L.; BADRI, M.; ST-YVES, D. Supporting predictive change impact analysis: a control call graph based technique. *12th Asia-Pacific Software Engineering Conference (APSEC'05)*, 2005. ISSN 1530-1362.
- BEIZER, B. *Software testing techniques*. [S.l.]: Van Nostrand Reinhold, 1990.
- BOHNER, S. Software change impacts-an evolving perspective. In: *Software Maintenance, 2002. Proceedings. International Conference on*. [S.l.: s.n.], 2002. p. 263–272. ISSN 1063-6773.
- BOHNER, S. A.; ARNOLD, R. S. Book; Book/Illustrated. *Software change impact analysis*. [S.l.]: Los Alamitos, Calif. : IEEE Computer Society Press, 1996. Includes bibliographical references (p. 361-374). ISBN 0818673842 (pbk.).
- BREECH, B.; TEGTMEYER, M.; POLLOCK, L. Integrating influence mechanisms into impact analysis for increased precision. In: *Software Maintenance, 2006. ICSM '06. 22nd IEEE International Conference on*. [S.l.: s.n.], 2006. p. 55–65. ISSN 1063-6773.
- CAI, H.; SANTELICES, R. Diver: Precise dynamic impact analysis using dependence-based trace pruning. In: *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering*. New York, NY, USA: ACM, 2014. (ASE '14), p. 343–348. ISBN 978-1-4503-3013-8. Disponível em: <<http://doi.acm.org/10.1145/2642937.2642950>>.

- CORNETT, S. Code Coverage Analysis. *Bullseye Testing Technology*, 2002.
- DAHL, O.-J.; DIJKSTRA, E. W.; HOARE, C. A. R. *Structured programming*. [S.l.]: Academic Press Ltd., 1972.
- ENGSTRÖM, E.; RUNESON, P.; SKOGLUND, M. A systematic review on regression test selection techniques. *Inf. Softw. Technol.*, Butterworth-Heinemann, Newton, MA, USA, v. 52, n. 1, p. 14–30, jan. 2010. ISSN 0950-5849. Disponível em: <<http://dx.doi.org/10.1016/j.infsof.2009.07.001>>.
- FEWSTER, M.; GRAHAM, D. *Software test automation: effective use of test execution tools*. [S.l.]: Addison-Wesley Reading, 1999.
- GROVE, D.; CHAMBERS, C. A framework for call graph construction algorithms. *ACM Trans. Program. Lang. Syst.*, ACM, New York, NY, USA, v. 23, n. 6, p. 685–746, nov. 2001. ISSN 0164-0925. Disponível em: <<http://doi.acm.org/10.1145/506315.506316>>.
- HIND, M.; PIOLI, A. Which pointer analysis should I use? *ACM SIGSOFT Software Engineering Notes*, v. 25, n. 5, p. 113–123, 2000. ISSN 01635948.
- IEEE. *IEEE Standard Glossary of Software Engineering Terminology*. 1990. 1-84 p.
- KALLIAMVAKOU, E. et al. The promises and perils of mining github. In: *Proceedings of the 11th Working Conference on Mining Software Repositories*. New York, NY, USA: ACM, 2014. (MSR 2014), p. 92–101. ISBN 978-1-4503-2863-0. Disponível em: <<http://doi.acm.org/10.1145/2597073.2597074>>.
- KOCHHAR, P. S.; THUNG, F.; LO, D. Code Coverage and Test Suite Effectiveness : Empirical Study with Real Bugs in Large Systems. p. 560–564, 2015.
- LAW, J.; ROTHERMEL, G. Whole program path-based dynamic impact analysis. *25th International Conference on Software Engineering, 2003. Proceedings.*, 2003. ISSN 0270-5257.
- LEHNERT, S. A review of software change impact analysis. *Ilmenau University of Technology, Tech. Rep*, 2011.
- LI, B. et al. A survey of code-based change impact analysis techniques. *Software Testing, Verification and Reliability*, v. 23, n. 8, p. 613–646, 2013. ISSN 1099-1689. Disponível em: <<http://dx.doi.org/10.1002/stvr.1475>>.
- MARINESCU, P.; HOSEK, P.; CADAR, C. Covrig: A Framework for the Analysis of Code, Test, and Coverage Evolution in Real Software. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*. San Jose, CA, USA: ACM, 2014. p. 93—104. ISBN 9781450326452. Disponível em: <<http://doi.acm.org/10.1145/2610384.2610419>>.
- MCGREGOR, J. D.; SYKES, D. A. *A practical guide to testing object-oriented software*. [S.l.]: Addison-Wesley Professional, 2001.
- MYERS, G. J.; SANDLER, C.; BADGETT, T. *The art of software testing*. [S.l.]: John Wiley & Sons, 2011.

- NTAFOS, S. A comparison of some structural testing strategies. *Software Engineering, IEEE Transactions on*, n. 8820975, p. 868–874, 1988.
- OOSTERWAAL, S. Combining source code and test coverage changes in pull requests. 2015.
- ORSO, A.; APIWATTANAPONG, T.; HARROLD, M. J. Leveraging field data for impact analysis and regression testing. *SIGSOFT Softw. Eng. Notes*, ACM, New York, NY, USA, v. 28, n. 5, p. 128–137, set. 2003. ISSN 0163-5948. Disponível em: <<http://doi.acm.org/10.1145/949952.940089>>.
- REN, X. *Change impact analysis for Java programs and applications*. [S.l.]: ProQuest, 2007.
- REN, X. et al. Chianti: A tool for change impact analysis of java programs. *SIGPLAN Not.*, ACM, New York, NY, USA, v. 39, n. 10, p. 432–448, out. 2004. ISSN 0362-1340. Disponível em: <<http://doi.acm.org/10.1145/1035292.1029012>>.
- RYDER, B. G.; TIP, F. Change impact analysis for object-oriented programs. In: *Proceedings of the 2001 ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering*. New York, NY, USA: ACM, 2001. (PASTE '01), p. 46–53. ISBN 1-58113-413-4. Disponível em: <<http://doi.acm.org/10.1145/379605.379661>>.
- SANTELICES, R. et al. Test-suite augmentation for evolving software. In: *Proceedings of the 2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. Washington, DC, USA: IEEE Computer Society, 2008. (ASE '08), p. 218–227. ISBN 978-1-4244-2187-9. Disponível em: <<http://dx.doi.org/10.1109/ASE.2008.32>>.
- SHAHID, M.; IBRAHIM, S. An Evaluation of Test Coverage Tools in Software Testing. v. 5, p. 216–222, 2011.
- SHAHID, M.; IBRAHIM, S. An evaluation of test coverage tools in software testing. In: *2011 International Conference on Telecommunication Technology and Applications Proc. of CSIT*. [S.l.: s.n.], 2011. v. 5.
- SOLTESZ, S. et al. Container-based operating system virtualization: a scalable, high-performance alternative to hypervisors. In: ACM. *ACM SIGOPS Operating Systems Review*. [S.l.], 2007. v. 41, n. 3, p. 275–287.
- SUN, X. et al. Change impact analysis based on a taxonomy of change types. In: *Computer Software and Applications Conference (COMPSAC), 2010 IEEE 34th Annual*. [S.l.: s.n.], 2010. p. 373–382. ISSN 0730-3157.
- SUN, X. et al. Using lattice of class and method dependence for change impact analysis of object oriented programs. In: *Proceedings of the 2011 ACM Symposium on Applied Computing*. New York, NY, USA: ACM, 2011. (SAC '11), p. 1439–1444. ISBN 978-1-4503-0113-8. Disponível em: <<http://doi.acm.org/10.1145/1982185.1982495>>.
- TIP, F. A survey of program slicing techniques. *JOURNAL OF PROGRAMMING LANGUAGES*, v. 3, p. 121–189, 1995.
- YANG, Q.; LI, J. J.; WEISS, D. M. A survey of coverage-based testing tools. *Computer Journal*, v. 52, n. 5, p. 589–597, 2009. ISSN 00104620.

ZHU, H.; HALL, P. a. V.; MAY, J. H. R. Software unit test coverage and adequacy. *ACM Computing Surveys*, v. 29, n. 4, p. 366–427, 1997. ISSN 03600300.

APÊNDICE A – Formato dos Fluxos de Execução

Amostragem do formato dos fluxos de execução obtidos como resultado na execução da ferramenta, onde o símbolo $>$ representa uma chamada de método, ou seja, o método anterior ao símbolo faz uma chamada ao método posterior ao símbolo. Os parâmetros dos métodos foram removidos para fins de formatação do texto.

1. SimpleCommandMap.dispatch() > AchievementCommand.execute() > Command.broadcastCommandMessage() > Command.broadcastCommandMessage()
2. SpawnpointCommand.execute() > Command.broadcastCommandMessage() > Command.broadcastCommandMessage()
3. StandardMessenger.validatePluginMessage() > ChannelNotRegisteredException.ChannelNotRegisteredException()
4. SimpleCommandMap.dispatch() > BanCommand.execute()
5. ChannelNotRegisteredException.ChannelNotRegisteredException() > ChannelNotRegisteredException.ChannelNotRegisteredException()
6. StandardMessenger.validatePluginMessage() > ChannelNotRegisteredException.ChannelNotRegisteredException()
7. SimpleCommandMap.dispatch() > DefaultGameModeCommand.execute()
8. SimpleCommandMap.dispatch() > GameModeCommand.execute()
9. PlayerPreLoginEvent.PlayerPreLoginEvent() > PlayerPreLoginEvent.PlayerPreLoginEvent()
10. SimpleCommandMap.dispatch() > HelpCommand.execute() > ChatPaginator.paginate()